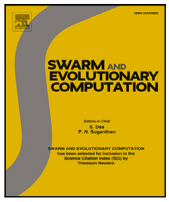
Swarm and Evolutionary Computation 82 (2023) 101361



Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo





A diversity-driven migration strategy for distributed evolutionary algorithms

Jean N.R. Araujo [a,*], Lucas S. Batista [a,b]

[a] Operations Research and Complex Systems Laboratory, Graduate Program in Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antonio Carlos 6627, 31270-901, Belo Horizonte - MG, Brazil

[b] Departamento de Engenharia Elétrica, Universidade Federal de Minas Gerais, Av. Antonio Carlos 6627, 31270-901, Belo Horizonte - MG, Brazil

ARTICLE INFO

Keywords:
Distributed evolutionary algorithm
Exploration and exploitation
Migratory policy
Migratory frequency
Online clustering algorithm
TEDA

ABSTRACT

The increasing complexity of real-world problems raises new challenges to evolutionary computation. Distributed models have been successfully employed by many evolutionary algorithms (EAs) to deal with these challenges. In particular, distributed models provide a means to enable collaboration between multiple subpopulations, thus allowing the design of strategies to deal with premature convergence and loss of diversity, which are common problems in traditional evolutionary algorithms. Through introducing periodic migrations, many Distributed Evolutionary Algorithms (DEAs) have been proposed to improve the balance between exploration and exploitation. However, most of them focus on performing migrations at fixed or probabilistic intervals. In this work, we present a mechanism to estimate the moment of executing the migrations by assessing the loss of diversity of the subpopulations. Another relevant issue is that most studies choose to migrate the best or a random individual. We report a strategy that identifies a migrant individual capable of generating diversity that helps a given subpopulation explore non-visited regions without harming its health. The proposed approach uses an online clustering algorithm to create clouds of good-fitness individuals that have been previously migrated. The solution to be migrated must be extracted from a cloud whose population distribution is sufficiently different from the population distribution of the original subpopulation. We called this approach a Diversity-driven Migration Strategy (DDMS). The efficiency of DDMS is experimentally compared against traditional migration strategies (fixed and probabilistic) on the CEC'2014 test suite. Considering the average error values for the objective function, the proposed approach is specially better in 50D and 100D (dimensional) instances. Regarding the diversity, the proposed strategy is better in 100% and about 96% of the test functions in 50D and 100D scenarios, respectively. In general, we have found the DDMS mechanisms to be of considerable benefit, especially in more complex problems such as the ones based on hybrid and composition objective functions.

1. Introduction

In recent years, several evolutionary algorithms (EAs) have emerged as efficient metaheuristics to deal with complex, multidimensional, non-differentiable, and multimodal optimization problems. Despite this prominent ability, EAs can suffer from premature convergence[1] [1]. To circumvent this issue, EAs have been adapted to run in distributed environments [2]. A common distributed policy is to break a population into small subpopulations. From there, each subpopulation evolves within an independent island, exploring different regions of the search space. Many distributed evolutionary algorithms (DEAs) employ a communication policy that involves migrating individuals between islands periodically. This exchange of information helps to promote convergence towards promising regions in addition to population diversity [2].

The literature has demonstrated that DEAs tend to outperform sequential EAs [2–6]. However, the performance of DEAs depends on some parameters, such as migratory frequency and policy. The migratory frequency concerns defining when to migrate. Generally, the higher the frequency, the faster the convergence. But, the algorithm may get trapped in local minimums [2]. On the other hand, when the migratory frequency is lower, the global exploration ability tends to be higher, but the algorithm can converge much slower [2]. This trade-off between exploration and exploitation demonstrates that the choice of an adaptive migration frequency is essential to achieve good

* Corresponding author.
E-mail address: jeannra@ufmg.br (J.N.R. Araujo).

[1] In this text, we use the concept of convergence to represent the state in which the population gets trapped in a local optimum, which can either be different or not from the known best one.

https://doi.org/10.1016/j.swevo.2023.101361
Received 20 January 2022; Received in revised form 11 March 2023; Accepted 5 July 2023
Available online 20 July 2023
2210-6502/© 2023 Elsevier B.V. All rights reserved.

performance. Nevertheless, most studies prefer to perform migrations at fixed or probabilistic intervals.

The migratory policy concerns choosing the individuals to replace and to be replaced when migration occurs. The most common policy is to send the best individual, which replaces a random individual on the neighboring island. However, such an approach can cause an increase in selective pressure since the subpopulations can become very similar over the generations [2]. A more suitable strategy involves migrating individuals that are not too similar to the neighboring island and, at the same time, have good fitness values.

With these issues in mind, the key motivations of this work are:

1. To propose an approach to estimate the instant to perform a migration based on the loss of subpopulation diversity. In this way, migrations are triggered according to stagnation and convergence information.
2. To propose a migratory policy to promote a *health* diversity in the subpopulation. The idea is to migrate an individual sufficiently different from those into the neighboring subpopulation. In addition, the proposed policy seeks to ensure the migrated individual has good fitness not to harm convergence. The strategy uses the *TEDA* (Typicality and Eccentricity Data Analysis) [7], which is a framework that incrementally models a non-parametric data distribution based on the cumulative proximity between data samples.

We named these two proposals a Diversity-driven Migration Strategy (DDMS). As key contributions, DDMS addresses both migratory frequency and migratory policy problems. Regarding the migratory frequency problem, migrations only occur when a loss of diversity is identified in the island. About the migratory policy problem in DDMS, the migrated individual must have: (i) good fitness and (ii) the potential to generate diversity on the target island.

The remainder of this paper is organized as follows. Section 2 presents a literature review on evolutionary algorithms that address the need for exploration and exploitation in the search space and shows how DEAs can be employed to improve the balance between exploration and exploitation. Section 3 describes the details of the DDMS. Section 4 provides and discusses the experimental results. Section 5 compares DDMS against some prominent sequential evolutionary algorithms. Section 6 provides a perspective on the time complexity of the proposed algorithms. Finally, we conclude this paper and present future work in Section 7.

2. Related work and motivation

The main challenge of evolutionary algorithms is to explore vast and promising regions of the search space while generating suitable solutions by converging to a particular area [1,8,9]. Countless applications have come across these questions: how to exploit without losing the ability to explore, and how to explore without losing the ability to exploit?

The authors in [10] point out that exploration is the process of visiting entirely new regions of a search space; on the other hand, exploitation is the process of visiting those regions of a search space within the neighborhood of previously visited points. However, this task is not trivial [10–16].

Generally, diversity can be used as an indirect measure to know the exploratory capacity of the population [10,15]. In the last instance, the lack of diversity drives the population finds itself stagnating at local optimum [10,13,14].

The authors in [17] highlight two situations in which evidence of performance deterioration can be perceived: stagnation and premature convergence. Stagnation occurs when the algorithm cannot generate better solutions for several successive generations. Premature convergence happens when the population gets trapped in a local minimum different from the known best one. In both cases, some strategies must be applied to improve diversity [17].

In addition, maintaining diversity must be driven to help to find out fitting individuals [10]. In this sense, the authors in [18] define the term "healthy diversity", which means trying to create and maintain a diverse population of highly fit (healthy) individuals capable of adapting quickly to landscape changes.

In this context, Distributed Evolutionary Algorithms (DEAs) can be used to improve the exploratory ability of a population while maintaining diversity. A DEA usually partitions the population into subpopulations as if they were islands. From there, each subpopulation evolves independently to explore different regions and avoid premature convergence.

A communication policy, called migration, can be implemented between the islands. This exchange of information is often used to promote convergence to promising regions and population diversity. The authors in [19–21] list some parameters that directly impact the island model performance. These parameters are summarized as follows:

- **Number of islands:** defines the number of subpopulations;
- **Migration topology:** describes the communication schema between the islands. A commonly adopted topology is the ring, in which each island has two neighboring islands, thus forming a ring-shaped topology;
- **Migratory rate:** defines how many individuals will be migrated from an island to another;
- **Synchronization type:** defines if the migration process is performed synchronously or asynchronously;
- **Migratory frequency:** defines the periodicity of the migration;
- **Migratory policy:** describes which individuals will be migrated and replaced when migration process takes place. The commonly used policies are *best-to-worst* and *best-to-random*, which stand for the best individual replacing the worst individual and the best individual replacing a random individual, respectively.

Some prominent studies in the literature focus on migratory frequency and policy since these strategies can be used directly to maintain or recovery diversity in island models or variants.

In [22], the authors propose to group the subpopulations into two families. The first one has the role of exploring the search space and is composed of three islands arranged according to a ring topology. The second one is composed of subpopulations with a population size that is progressively reduced. The idea is that the subpopulations belonging to the second family are highly exploitative. The solutions generated by the second family then migrate to the first family. Instead of fixed-interval migrations, a copy of the best individual in the subpopulation is sent to the next island in the ring with a given probability. An incoming individual replaces a random local individual.

The authors in [23] discuss the importance of defining a suitable migratory policy. They present a Genetic Algorithm (GA) that uses a diversity-driven approach that chooses the individuals to be sent to other islands based on the principle of *multiculturality*, that is, the individual migrated should be different enough from the target population. They demonstrate that such a policy outperforms the standard policy of sending the best or a random individual. However, the authors fixedly define the migratory frequency.

In [6], the authors propose a simple distributed island-based Differential Algorithm (DE) in which the population is partitioned into two identical subpopulations. The subpopulations evolve independently with the same parameters, and after 100 generations, one migration is made to promote diversity. In migration, one randomly selected individual is sent from one of the islands to another. Incoming individual replaces randomly chosen local individual only if the former is better. This approach effectively improves global search capability when compared to traditional DE. On the other hand, migration in a fixed interval has some drawbacks. In [3], the authors demonstrate that can occur

a trade-off between exploration and exploitation when using different migration frequencies. A higher migratory frequency can speed up the convergence, but the algorithm may get trapped. Otherwise, when the migratory frequency is lower, the algorithm exhibits better global exploration capability but tends to converge much more slowly.

In [24], the authors use a *GA* to implement a dynamic island model and a new migration schema based on similarity. The model starts with a single island in the first generation. When the evolution reaches a specific iteration, a spectral clustering algorithm is introduced to split the population into a set of new islands. Similar individuals are assigned to the same island. Then, each new island evolves independently. Every 50 generations, all individuals migrate to a pool wherein the spectral clustering algorithm is executed to reconstruct the islands according to the difference-based similarities of the individuals. After several generations, the best individuals are considered as potential global optimized solutions.

In [25], the authors combine two strategies to preserve diversity in the island model. The first strategy uses different kinds of GAs for each island to force the generation of different individual characteristics and slow down the global diversity among the islands. The second strategy implements a novel migration policy that splits migrants into two classes: pursuer and avoider. An avoider has a task to promote diversity on the island, while a pursuer tries to enable exploration. A migrant of an island has a certain probability of being a pursuer or avoider, depending on the island's current state. This mechanism seeks to ensure the individual migrates to the correct island dynamically, which will maintain the diversity. The authors show that combining these strategies has great potential to preserve the overall island diversity.

The authors in [26] propose a new alternative to implement the island model, called the Stigmergy Island Model (Stgm-IM), inspired by the natural phenomenon of stigmergy. In this model, the population of each island is evolved by a distinct EA, which can imply different evolutionary characteristics applied in parallel to solve the problem. Each connection between the islands is dynamically weighted adaptively, according to the attractiveness between each pair of islands. The idea is that the islands with the most suitable EAs to solve the problem become more attractive, and more solutions are directed to them.

In [27], the authors propose a new hybrid architecture called Parallel Hybrid Island (PHI). In this model, each island corresponds to an algorithm that computes independently and then shares the information during the migration operation (it occurs for every two iterations). Combining Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Fireworks Algorithm (FWA), this strategy of the hybridization of populations using various algorithms can improve population diversity because these distinct algorithms can have different evolution trails.

These studies have demonstrated the potential of DEAs to address a wide variety of optimization problems. In this work, we consider migration as a tool to balance exploration and exploitation adaptively. To this end, we propose some strategies to define the migratory frequency and policy, as summarized below.

1. *Migratory frequency*. Most studies focus on performing migrations at fixed or probabilistic intervals [3,6,23,24,28,29]. We propose migration be triggered according to some measure that reports a possible loss of diversity. The key motivation is to try to avoid unnecessary migrations.
2. *Migratory policy*. Most studies focus on migrating either the best or a random individual [3,6,24,27,28,30,31]. We propose the migrated individual be sufficiently different from the ones of the target subpopulation distribution to improve the exploration. Besides, we suggest a method to ensure the migrated individual has good fitness not to harm convergence.

3. Diversity-driven migration strategy

JADE and *SHADE* are good examples of evolutionary algorithms that achieve excellent performance when promoting exploration and exploitation. In [32], the authors have shown *JADE/SHADE*-based EAs outperform the vast majority of other metaheuristics (e.g., *DE*, *PSO*, *GA*, and variants) on some benchmark functions and real-world problems. Despite excellent results, these algorithms tend to lose diversity in the final stages of evolution [15,33] or when the problem has many variables because of the curse of dimensionality [34,35].

We are interested in introducing a distributed structure in which the migration strategy involves defining: (i) the moment to carry out the migrations and (ii) the composition of the individuals to be migrated. In this section, we present a new diversity-driven migration mechanism according these two propositions.

3.1. Assessing the loss of diversity

In distributed algorithms, migration can be unnecessary or even out-of-time when done without criteria. A reasonable scenario would be diversify when the population converges prematurely or stagnates.

3.1.1. Measuring premature convergence

In [17], the authors propose a method to measure the premature convergence, as follows:

$$\mu_j^g = \frac{1}{NP}\sum_{i=1}^{NP} x_{i,j}^g \tag{1}$$

$$\sigma_j^g = \sqrt{\frac{1}{NP}\sum_{i=1}^{NP}(x_{i,j}^g - \mu_j^g)^2} \tag{2}$$

in which NP is the population size, μ_j^g and σ_j^g are the mean and the standard deviation of the population in the jth dimension at generation g, respectively. The standard deviation (σ_j^g) measures the population diversity in the jth dimension. When $\sigma_j^g \approx 0$, we have a strong indication that the jth dimension has lost diversity. To denote whether the population has converged in the jth dimension at the gth generation, the flag $\bar{\tau}_j^g$ is defined as follows:

$$\bar{\tau}_j^g = \begin{cases} 1, & \text{if } \sigma_j^g \leq \omega_j^g \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

If σ_j^g is not greater than ω_j^g, $\bar{\tau}_j^g$ gets 1 to indicate the population has converged in the jth dimension. The diversity is often large at the beginning of the evolutionary process, and consequently, the σ_j^g value is also large. Despite being problem-dependent, it usually takes a long time for the σ_j^g to drop to zero. Therefore, ω_j^g assumes a small value based on (4).

$$\omega_j^g = \min(T, \theta_j^g) \tag{4}$$

The parameter T assumes a value close to zero (e.g., 10^{-3}). A variable θ_j^g is added to assess whether the jth dimension had already converged on that same region in previous generations. The value of θ_j^g is defined as follows:

$$\theta_j^g = \begin{cases} |\mu_j^g - \mu_j^{g^-}| \times T, & \text{if } \sigma_j^g \leq T \\ T, & \text{otherwise.} \end{cases} \tag{5}$$

Suppose the population has converged in a given generation before g, which we can call generation g^-. Therefore, $\boldsymbol{\mu}^{g^-}$ vector is the population mean in generation g^-, regarding all subpopulations. If $\sigma_j^g \leq T$, the population has converged in the jth dimension in the current generation g. We are interested in finding if this convergence in dimension j occurs in the same region as generation g^-. If $|\mu_j^g - \mu_j^{g^-}|$ is small, we assume that the population has converged to the same region. Observe in (5) that, in this case, θ_j^g is set to $|\mu_j^g - \mu_j^{g^-}| \times T$. Thus, as θ_j^g is

less than T, ω_j^g receives θ_j^g in (4). Note that while the algorithm remains converging to the same local minimum even after applying some diversification mechanism, ω_j^g value tends to decrease. Consequently, σ_j^g must be less than $\sigma_j^{g^-}$ to consider that the jth dimension converged in (3). This schema allows the population to sufficiently exploit a revisited local optimum before using any diversification mechanism.

3.1.2. Measuring stagnation

Stagnation occurs when the population distribution does not change significantly for a while. This fact can be identified while the mean and the standard deviation remain unchanged for some successive generations [17]. The following variable λ_j^g denotes the number of successive generations in which μ_j^g and σ_j^g remain unchanged for the jth dimension at the gth generation. It is calculated as follows:

$$\lambda_j^g = \begin{cases} \lambda_j^{g-1} + 1, & \text{if } \mu_j^g = \mu_j^{g-1} \text{ and } \sigma_j^g = \sigma_j^{g-1} \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

in which $\lambda_j^g = 0$ in first generation. Another flag $\hat{\tau}_j^g$ is defined to denote when the population has stagnated in the jth dimension at the gth generation, as follows:

$$\hat{\tau}_j^g = \begin{cases} 1, & \text{if } \lambda_j^g \geq \text{UN} \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

in which UN is an integer value. If $\lambda_j^g \geq \text{UN}$ then $\hat{\tau}_j^g = 1$, which indicates that the population can be stagnated in the jth dimension. In [17], the authors show the larger the population size, the more generations it will take for the population to enter a stable stagnation state. Therefore, they use UN = NP, in which NP is the population size.

The authors in [17] define that if $\bar{\tau}_j$ or $\hat{\tau}_j$ is equal to 1 (see (8)), the jth dimension needs of diversity. In this work, we use migrations between islands to deal with such a loss of diversity.

$$\tau_j^g = \begin{cases} 1, & \text{if } \bar{\tau}_j^g = 1 \text{ or } \hat{\tau}_{j,g} = 1 \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

3.2. Determining migratory frequency

After identifying whether the population has converged or stagnated in the jth dimension, some rules are checked to decide if migration should be triggered.

- **Rule 1:** Migration occurs when all dimensions have lost diversity, as shown below [17]:

$$\Gamma 1^g = \begin{cases} 1, & \text{if NDIV} = \text{D} \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

in which NDIV is the number of dimensions that need some diversity, being calculated as $\text{NDIV} = \sum_{j=1}^{\text{D}} \tau_j^g$.

- **Rule 2:** For high-dimensional problems, a large number of function evaluations (generations) would be required for the algorithm to trigger **Rule 1** ($\Gamma 1^g$). To alleviate this issue, migration occurs with a small probability in any dimension j according to the following condition [17]:

$$\Gamma 2^g = \begin{cases} 1, & \text{if } \tau_j^g = 1 \quad \text{and} \quad rand() < c \\ 0, & \text{otherwise} \end{cases} \tag{10}$$

in which c is a small constant number, e.g., $c = 10^{-3}$ [17]. The $rand()$ function generates a uniformly distributed random number within $[0, 1]$.

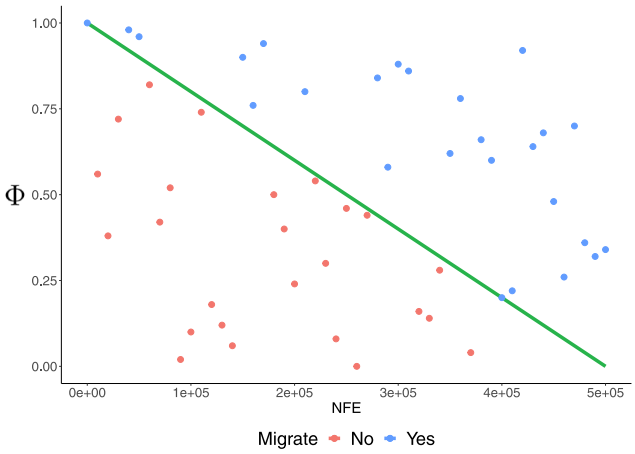


Fig. 1. Rule 3: When to migrate. Green line represents the exponential probability distribution. Red points are generations in which a migration must not occur, and blue points are generations in which a migration must occur.

- **Rule 3:** In a distributed algorithm, inopportune migrations happen because most proposals use fixed or probabilistic migrations. For this reason, we propose a third rule, as shown below:

$$\Gamma 3^g = \begin{cases} 1, & \text{if } \Phi \geq \Psi \\ 0, & \text{otherwise} \end{cases} \tag{11}$$

in which

$$\Phi = \frac{\text{NDIV}}{\text{D}} \quad \text{and} \quad \Psi = 1 - \left(\frac{\text{NFE}}{\text{NFE}^{max}} \right). \tag{12}$$

In this definition, D is the number of decision variables, and NDIV is the number of dimensions that need some diversity. Then, Φ represents the proportion of dimensions that must be diversified. NFE is the current number of function evaluations, while NFE^{max} is the maximum number of function evaluations. We define a metric that helps to estimate when diversity is least likely to be lost. Diversity loss is much less likely to occur early in the evolutionary process, so the value of Ψ is higher at this stage. Therefore, migrations only happen early in the evolutionary process if the value of Φ is massive.

As the algorithm evolves, the loss of diversity becomes more likely to happen, so Ψ becomes smaller. Thus, more migrations can occur at the end of the evolutionary process, even if the Φ value is small.

Fig. 1 shows that this approach can identify situations of diversification that the authors in [17] do not cover. The green line represents the Ψ values as the algorithm evolves. The points are the proportion of dimensions that must be diversified (Φ) in that generation.[2] The red points are generations in which a migration must not occur, and the blue points are generations in which a migration must occur, according to the **Rule 3** ($\Gamma 3^g$).

According to this proposal, migration can occur in the first half of evolution since the proportion of variables that must be diversified (Φ) is not too small. On the other hand, the probability for dimensions to converge or stagnate increases after half the evolution. Thus, even with the small value of Φ, more migrations can occur to improve diversity at the end of the evolutionary process.

Eq. (13) summarizes the proposed rule for determining when to migrate.

$$\Gamma^g = \begin{cases} 1, & \text{if } \Gamma 1^g = 1 \text{ or } \Gamma 2^g = 1 \text{ or } \Gamma 3^g = 1 \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

[2] For simplicity, we generate these points randomly.

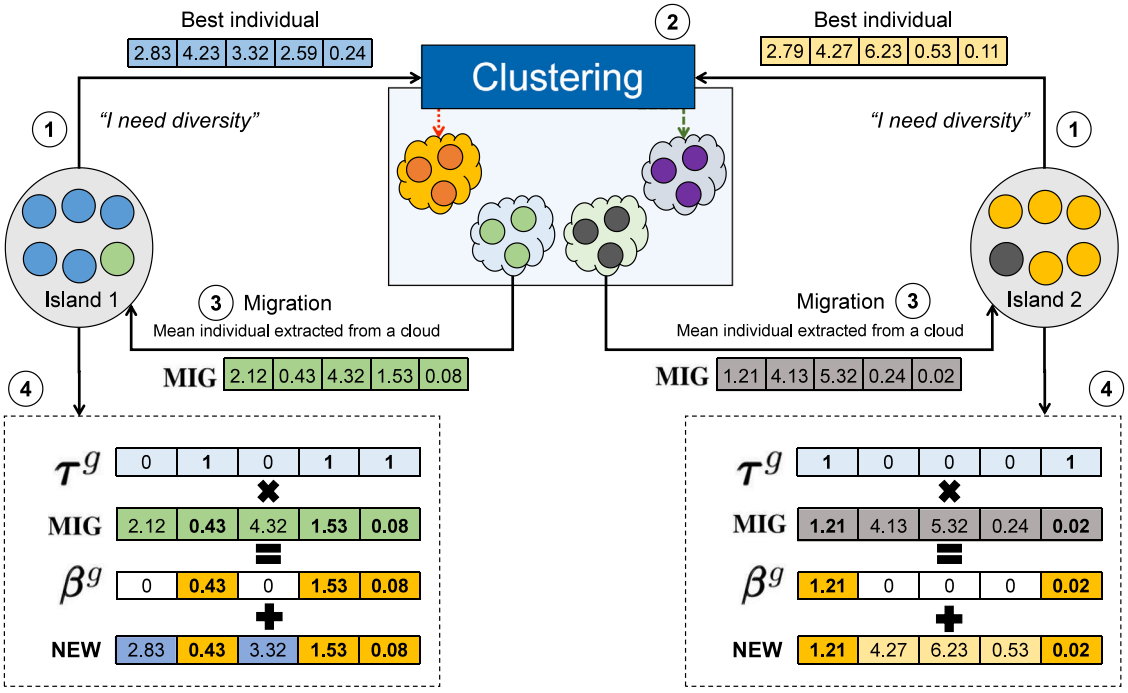


Fig. 2. Illustration of a distributed EA with the proposed migratory policy. Circles into the island represent the individuals of the population. The values in the small circles and the black arrows represent the steps/sequence of the events. The dashed rectangles show actions taken into the islands. The rectangle with a straight line houses the clouds obtained by the clustering algorithm.

Thus, after each generation g, this procedure generates the Γ^g value. If $\Gamma^g = 1$, then the migration must happen; otherwise, the migration is discarded.

3.3. Defining migratory policy

From now on, we are interested in defining how migration should be done. Fig. 2 illustrates the proposed strategy. The idea is to migrate an individual not too similar to the island population that has lost diversity (target island). Meanwhile, we seek to generate multiple migration options considering the recent migration history.

Consider that Islands 1 and 2 identify the need for migration according to some of the rules presented. In this case, they send a vector with the best individual to a pool (step 1). In this pool, such a best individual will be clustered into a cloud together with similar individuals.

After grouping, the algorithm chooses an individual to be migrated (**MIG**). Preferably, this individual should be different from the population distribution of the target island (step 2). Subsequently, **MIG** is then sent to the target island (step 3). Finally, the target island selects the values in the variables that should be diversified considering the $\boldsymbol{\tau}^g$ vector (step 4). Next, the empty dimensions of $\boldsymbol{\beta}^g$ are filled regarding the best individual of the island, then generating a new individual (**NEW**) that replaces a random individual into the subpopulation.

In contrast to traditional migration models that involve direct communication policies, this proposal introduces an island model based on a clustering framework. This practice can provide straight control of exploration and exploitation to maintain diversity. To this end, we seek to increase the probability that the migrated individual will be valuable in promoting diversity. Meanwhile, the migrated individual must have a quality track record, which help the target island escape stagnation and move into promising new regions.

3.4. Clustering algorithm

In [36], the authors introduce a new evolving clustering algorithm called *TEDA*-Cloud based on *TEDA* (Typicality and Eccentricity Data Analysis) algorithm [7]. Essentially, they implement a clustering algorithm that evolves iteratively and autonomously without the need for constant adjustments of its parameters.

The main advantage of this approach over other clustering approaches is its online, incremental, and recursive nature, which makes it light and flexible to model in real-time the changes of individuals within each island [36,37].

3.4.1. TEDA

TEDA is a framework proposed in [7] which is specially used to detect an anomaly. *TEDA* incrementally models a non-parametric data distribution based only on the cumulative proximity between data samples.

Consider a D-dimensional input vector $\mathbf{x}^k$ in the timestamp k. The cumulative proximity $\pi(\cdot)$ of $\mathbf{x}^k$ concerning all existing data samples is calculated as:

$$\pi^k(\mathbf{x}^k) = \sum_{i=1}^{k} \text{dist}(\mathbf{x}^k, \mathbf{x}^i) \tag{14}$$

in which $\text{dist}(\mathbf{x}^k, \mathbf{x}^i)$ is the distance between data points $\mathbf{x}^k$ and $\mathbf{x}^i$; and k is the timestamp when the data point $\mathbf{x}$ is sampled. For anomaly detection, *TEDA* works with two concepts: eccentricity and typicality. Eccentricity is a measure of the dissimilarity between the data point $\mathbf{x}^k$ and all data points received up to the timestamp k. On the contrary, the typicality represents how typical an arbitrary data point $\mathbf{x}^k$ is concerning all the data points received until the timestamp k. For the case of Euclidean distance, eccentricity can be calculated recursively as follows [7]:

$$\xi^k(\mathbf{x}^k) = \frac{1}{k} + \frac{(\boldsymbol{\mu}^k - \mathbf{x}^k)^T(\boldsymbol{\mu}^k - \mathbf{x}^k)}{k \times (\sigma^2)^k} \tag{15}$$

in which $\boldsymbol{\mu}^k$ and $(\sigma^2)^k$ are the mean and variance, respectively, which can also be recursively updated:

$$\boldsymbol{\mu}^k = \frac{k-1}{k} \times \boldsymbol{\mu}^{k-1} + \frac{\mathbf{x}^k}{k} \tag{16}$$

$$(\sigma^2)^k = \frac{k-1}{k} \times (\sigma^2)^{k-1} + \frac{1}{k-1} \times \|\boldsymbol{\mu}^k - \mathbf{x}^k\|^2 \tag{17}$$

with $k \geq 1$. When $k = 1$, $\boldsymbol{\mu}^k = \mathbf{x}^k$ and $(\sigma^2)^k = 0$. The normalized eccentricity $\zeta^k(\mathbf{x}^k)$ and typicality $T^k(\mathbf{x}^k)$ can be obtained as follows:

$$\zeta^k(\mathbf{x}^k) = \frac{\xi^k(\mathbf{x}^k)}{2} \tag{18}$$

$$T^k(\mathbf{x}^k) = \frac{1 - \xi^k(\mathbf{x}^k)}{k-2} \tag{19}$$

The normalized eccentricity $\zeta^k(\mathbf{x}^k)$ can be used to determine if $\mathbf{x}^k$ is an outlier concerning the data set. For this purpose, $\zeta^k(\mathbf{x}^k)$ becomes a threshold based on the well known *Chebyshev* inequality [38]:

$$\zeta^k(\mathbf{x}^k) > \frac{m^2+1}{2 \times k} \tag{20}$$

in which m is a constant value greater than 0. This inequality can be explained in the following way: if the actual sample $\mathbf{x}^k$ is $m \times \sigma$ away from the mean, then it is considered an outlier concerning the data set. Therefore, m represents how many standard deviations distant from the mean a data sample should be to be considered an outlier [7]. We use the robustness to anomaly detection of *TEDA* to extract the individual that will be migrated.

3.4.2. Adapted TEDA -cloud

We create and update clusters based on the *TEDA*-Cloud algorithm proposed in [36]. The approach generates granular data structures called *data clouds*, whose main characteristic is that they do not have predefined shapes or boundaries as traditional data clusters. In *TEDA*-Cloud, *data clouds* are being formed as the data arrives by considering common properties among the data samples. This way, the set of *data clouds* directly describes all previous data samples. Another characteristic of *TEDA*-Cloud is the possibility to represent the data sets in terms of fuzzy membership. An example is that a particular data sample can belong to all *data clouds* with different membership degrees, with values between $[0, 1]$.

The algorithm derives each equation from *TEDA* to a generalized form, in which each *data cloud* is an independent data set. Then, it determines the membership of each read data sample to each existing *data cloud*, based on (20). Let $DC_i, i = 1, \ldots,$NC, be the set of *data clouds*, in which NC is the number of clusters. A prototype of DC_i is defined by the following parameters, which are updated every time a new data point arrives at the pool:

- s_i^k: number of data samples;
- $\boldsymbol{\mu}_i^k$: the mean, which represents the center of *data cloud*;
- $(\sigma^2)_i^k$: variance;
- $\xi_i^k(\mathbf{x}^k)$, $\zeta_i^k(\mathbf{x}^k)$: eccentricity and normalized eccentricity;
- $T_i^k(\mathbf{x}^k)$: normalized typicality;

The first cluster DC_1 is created when the first data sample $\mathbf{x}_1$ arrives. In this case, the parameters are initialized as follows:

$$\mathtt{NC} = 1, \quad s_1^1 = 1, \quad \boldsymbol{\mu}_1^1 = \mathbf{x}_1, \quad (\sigma^2)_1^1 = 0 \tag{21}$$

It is worth mentioning that typicality and eccentricity can only be calculated with at least two data samples.

We adapt *TEDA*-Cloud to work into a pool that receives the best individual from a given island whenever a migration is triggered, according to the rules previously presented. Basically, for each individual $\mathbf{x}^k$ which arrives at the pool, *TEDA*-Cloud computes its eccentricity $\zeta_i^k(\mathbf{x}^k)$ in relation to the *data cloud* DC_i. If $\zeta_i^k(\mathbf{x}^k)$ is high, we assume that $\mathbf{x}^k$ is considerably different from the data samples belonging to DC_i. Therefore, it is not necessary to perform updates to the cloud structure. Otherwise, we assume that $\mathbf{x}^k$ is similar to the data samples belonging to DC_i, hence it is necessary to update the number of points (s_i^k), the mean ($\boldsymbol{\mu}_i^k$), and the variance $(\sigma^2)_i^k$ of the cloud structure. Now, if $\mathbf{x}^k$ is considerably different from all existing *data clouds*, a new cloud is created. Thus, one of the conditions below may occur [36]:

- **Condition 1**: $\mathbf{x}^k$ is not an outlier for at least one cluster; then update all the clusters for which this condition holds according to the following generalized versions of the *TEDA* equations:

$$\begin{aligned}
s_i^k &= s_i^{k-1} + 1 \\
\boldsymbol{\mu}_i^k &= \frac{s_i^k - 1}{s_i^k} \times \boldsymbol{\mu}_i^{k-1} + \frac{\mathbf{x}^k}{s_i^k} \\
(\sigma^2)_i^k &= \frac{s_i^k - 1}{s_i^k} \times (\sigma^2)_i^k + \frac{1}{s_i^k - 1} \times \left(\frac{2 \times \|\mathbf{x}^k - \boldsymbol{\mu}_i^k\|}{\mathtt{D}} \right)^2 \\
\xi_i^k(\mathbf{x}^k) &= \frac{1}{s_i^k} + \frac{2 \times (\boldsymbol{\mu}_i^k - \mathbf{x}^k)_T (\boldsymbol{\mu}_i^k - \mathbf{x}^k)}{s_i^k \times (\sigma^2)_i^k \times \mathtt{D}} \\
\zeta_i^k(\mathbf{x}^k) &= \frac{\xi_i^k(\mathbf{x}^k)}{2}
\end{aligned} \tag{22}$$

  in which D is the dimensionality of the data set.
- **Condition 2**: $\mathbf{x}^k$ is an outlier for all existing clusters; then create a new cluster with the following parameters:

$$\mathtt{NC} = \mathtt{NC} + 1 \quad s_{\mathtt{NC}}^k = 1 \quad \boldsymbol{\mu}_{\mathtt{NC}}^k = \mathbf{x}^k \quad (\sigma^2)_{\mathtt{NC}}^k = 0. \tag{23}$$

Similar to *TEDA*, the condition to define if a data point $\mathbf{x}^k$ is an outlier in relation to a *data cloud* DC_i is given by:

$$\zeta_i^k(\mathbf{x}^k) > \frac{m^2+1}{2 \times s_i^k}. \tag{24}$$

However, the authors in [37] argue that the condition expressed by Eq. (24), with $m \geq 1$, will always be false when $s_i^k = 2$. I other words, the second data point of the cluster DC_i will never be considered an outlier. This fact can be a problem, especially when the first two data points are far from each other. In practice, massive clusters can be generated and, as a consequence, dense regions can be modeled incorrectly in the data space. Thus, the authors in [37] added a parameter r_0 to limit the variance of each cluster when $s_i^k = 2$ to prevent a cluster from growing indefinitely. This parameter modifies the outlier condition expressed by Eq. (24) as follows:

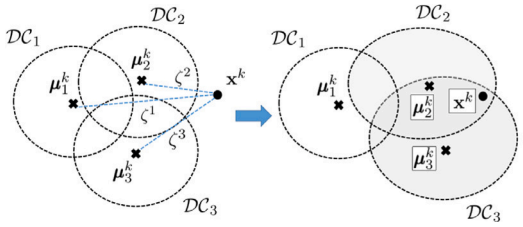
$$\zeta_2^i(\mathbf{x}_2) > \frac{m^2+1}{4} \quad \text{and} \quad (\sigma_2^i)^2 < r_0. \tag{25}$$

with $r_0 = 0.001$ [37].

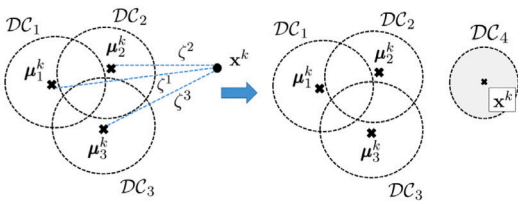
Figs. 3(a) and 3(b) illustrate these two conditions by showing three data clouds, DC_1, DC_2 and DC_3, and an input data sample $\mathbf{x}^k$ at the kth time instant. In Fig. 3(a), *TEDA*-Cloud calculates the normalized eccentricities of $\mathbf{x}^k$ ($\zeta_i^k(\mathbf{x}^k)$) in relation to all three *data clouds*. As $\mathbf{x}^k$ belongs to DC_2 and DC_3, these two *data clouds* are updated, while DC_1 is kept unchanged. If the second condition is met as it illustrated in Fig. 3(b), a cloud $DC_{\mathtt{NC}+1}$ is created with the parameters present in Eq. (23). Note that $\mathbf{x}^k$ is a point considerably distant from all existing *data clouds*.

To adjust the *TEDA*-Cloud to the distributed model, we propose some adaptations, as listed below.

1. **Storage window of individuals:** As a recursive algorithm, *TEDA* does not need to store the data in memory, as only three statistical resources are required for each *data cloud* DC_i: s_i^k, $\boldsymbol{\mu}_i^k$ and $(\sigma^2)_i^k$ [36]. However, we need to migrate an individual to the target island. In our approach, we send the mean individual considering a given number of best individuals of the cloud. Besides that, we employ a restart mechanism if there is evidence that all *data clouds* are in the same search region. Therefore, we have established a maximum number w of individuals that can be saved in a *data cloud*. If the last individual $\mathbf{x}^k$ is not an outlier, it must be inserted in the *data cloud*. If the cloud is complete, we define a sliding window scheme in which the oldest individual is discarded, the window slides, and the individual $\mathbf{x}^k$ is inserted in the first position of the window.



(a) *Data clouds* updating: left illustration shows $\mathcal{DC}_1$, $\mathcal{DC}_2$, $\mathcal{DC}_3$ and a newly arrived data sample $\mathbf{x}^k$; right illustration shows the clouds after updating.



(b) *Data clouds* creating: left illustration shows $\mathcal{DC}_1$, $\mathcal{DC}_2$, $\mathcal{DC}_3$ and a newly arrived data sample $\mathbf{x}^k$; right illustration shows the clouds after creation of $\mathcal{DC}_4$.

Fig. 3. Updating and creating conditions of the *data clouds*.

2. **Restart mechanism:** Although the strategy of generating *data clouds* increases migration options, it is still possible that all clouds have similar individuals at some point in evolution. When this occurs, we have an indication that the islands are exploring the same search region, which can make migration unproductive. An excellent way to find if this has occurred is when an incoming individual is not an outlier in any of the *data clouds*, i.e., for all *data clouds*. To circumvent this limitation, we propose a cloud restart mechanism when such a condition occurs. The idea is to perform a perturbation in the *data cloud* $\mathcal{DC}_i$, according to (26):

$$\mathbf{x}_i = \mathcal{N}(\boldsymbol{\mu}_i, \sigma) + (\epsilon \times \mathbf{x}_{best} - \bar{\epsilon} \times \mathbf{x}_i) \tag{26}$$

in which ϵ and $\bar{\epsilon}$ are two uniform random numbers in the range $[0, 1]$, $\mathbf{x}_{best}$ is the best individual among the w individuals from $\mathcal{DC}_i$, $\mathbf{x}_i$ is ith individual among the w individuals from $\mathcal{DC}_i$. $\boldsymbol{\mu}_i$ and σ are the mean and the standard deviation of $\mathcal{DC}_i$, respectively. This perturbation only occurs when the number of clouds is greater than 1.

3. **Number of clouds:** It is important to stress that the goal of using the pool is to generate multiple migration options taking into account the migration history. Therefore, we are interested in generating a sufficient number of *data clouds* to provide some diversity. So, we have used a parameter NC_{max} to define the maximum number of *data clouds*. If the incoming individual is an outlier to all *data clouds* and $\text{NC} = \text{NC}_{max}$, this individual is added into an existing random cloud $\mathcal{DC}_{rand}$, observing the issue of the storage window of individuals. This strategy is also suitable for generating diversity within the clouds.

Since clustering has been performed, we need to choose the individual that should migrate to the target island. For this purpose, we select a cloud $\mathcal{DC}_{rand}$ randomly among those where $\mathbf{x}^k$ is an outlier. After that, we send the $\mathcal{DC}_{rand}$ mean individual, $\boldsymbol{\mu}$, to the target island. Thus, we seek to ensure that the migrated individual will generate diversity in the target island.

3.5. General structure of the DDMS

Algorithm 1 presents the pseudo-code of the DDMS. The algorithm has as input parameters: the maximum number of function evaluations (NFE^{max}); the size of population (NP); the number of dimensions (D); and the *role*, which can be "pool" or "island". The subpopulation is initialized randomly, and an objective function (f) evaluates solutions (line 1). At any given iteration (line 3), the algorithm executes until met the stopping condition. Generally, the algorithm stops when NFE^{max} is reached or the optimal solution is found. In each generation g, the number of function evaluations (NFE) is computed according to the number of individuals in the population (line 4).

A typical island runs an optimizer that guides the evolutionary process (line 6). In this work, we choose *L-SHADE* as optimizer [39] because it is a classic evolutionary algorithm with excellent performance in the literature. After that, the island identifies in which dimensions the population has lost diversity, as shown in Section 3.1 (lines 7). Finally, the rules described in Section 3.2 are applied to define if a migration must occur or not (line 8). If a migration must occur (line 10), the island sends its best individual to the pool (line 11).

In a loop, the pool is always available to receive the best individual (**best_ind**) of any island that needs diversity (line 17). In this case, the clustering algorithm presented in Section 3.4.2 is used to group such individuals in *data clouds* (line 18). After that, the mean individual (**MIG**) is extracted from any *data cloud* in which the **best_ind** is considered an outlier (line 19). Subsequently, this selected individual (**MIG**) is then sent to the target island (line 20).

Once the migrated individual (**MIG**) has been received (line 12), the target island selects the values in the dimensions that should be diversified, according to the anomaly detection of *TEDA* described in Section 3.4.1. The empty dimensions of $\boldsymbol{\beta}^g$ are filled by the best individual, and this new individual replaces a random solution, different from the best one (lines 13–14). Lastly, g is incremented, and a new generation begins (line 21).

4. Computational experiments

Thirty test instances, proposed in the CEC'2014 special session and competition on single-objective real-parameter numerical optimization, were used to study the performance of the proposed strategy. A detailed description of these test instances can be found in [40]. In this benchmark, $f_1 - f_3$ are unimodal functions (U), $f_4 - f_{16}$ are simple multimodal functions (SM), $f_{17} - f_{22}$ are hybrid functions (H), and $f_{23} - f_{30}$ are composition functions (C).

We have compared the diversity-driven migration strategy (DDMS) with some prominent sequential (non-distributed) evolutionary algorithms, namely *SHADE* [41], *L-SHADE* [39], and *SHADE-ILS* [42].

SHADE (Success-History based Adaptive DE) [41] is an improved version of *JADE* [43] which uses a different parameter adaptation mechanism. In short, *SHADE* maintains a diverse set of parameters to drive control parameter adaptation. *L-SHADE* is an improvement of *SHADE*. The authors in [39] incorporate the Linear Population Size Reduction (LPSR) framework [44], which reduces the population linearly as a function of the number of function evaluations. *L-SHADE* is a prominent algorithm and is well known for its excellent performance in competitions [32]. *SHADE-ILS* [42] is a more recent improved version of *SHADE* that incorporates an Iterative Local Search that combines the exploration power of a contemporary DE algorithm with the exploitation ability of several local search methods.

The objective is to understand if the proposed distributed approach has the potential to tie or surpass these outstanding non-distributed evolutionary algorithms.

We also compared DDMS against traditional migration strategies in terms of convergence and diversity. To do this, we use two approaches, called FIX (fixed-interval migrations) and PROB (probabilistic migra-

Algorithm 1 General structure of the DDMS.

NFE^{max}: Maximum number of function evaluations; NP: size of population; D: Number of dimensions; *role*: define the role of the island;

```
Initialize subpopulation with NP individuals
g ← 1                                                          ▷ First generation
while the stopping condition is not met do
    NFE ← g × NP                                               ▷ Number of function evaluations
    if role == "island" then                                   ▷ Island
        Execute optimizer
        Identify premature convergence and stagnation using equations (3) and (7)
        Determine if a migration must occur using equation (13)

        if migration must occur then                           ▷ "Need diversity"
            Send the best individual, best_ind, to the pool
            Receive the migrated individual MIG
            Store in β_g the dimensions that should be diversified
            Replace a random individual by composition of β_g and best_ind
    else if role == "pool" then                                ▷ Pool
        while migration request has arrived do
            Receive the best individual, best_ind, from target island
            Perform clustering of best_ind
            Extract the mean individual, MIG, of one of the clouds in which best_ind is an outlier
            Send MIG to the target island
    g ← g + 1                                                  ▷ Update generation
```

tions). The main objective is to understand if migration strategies have any significant impact on the performance of the optimizer that runs on each island.

For each algorithm and each test function, 30 independent runs were conducted with $NFE^{max} = 10000 \times D$ evaluations as the termination criterion, in which D is the number of dimensions. When the objective function gap between the best found solution and the optimal one is 10^{-8} or less, the error (score) is treated as 0, and the algorithm terminates.

The parameter settings of *SHADE*, *L-SHADE* and *SHADE-ILS* are the same as those used in their original papers [39,41,42]. The control parameters of *TEDA/TEDA*-Cloud are: (1) the number of standard deviations distant from the mean for a data to be considered an outlier, m; (2) the maximum number of *data clouds*, NC^{max}; (3) the window size for storing individuals in *data clouds*, w. With $m = 1$, we are not rigorous when evaluating whether a point is an outlier concerning a given data distribution. So it is possible to generate multiple cluster options. Regarding NC^{max} and w parameters, we are interested in having several cluster options and a sufficient number of individuals within the clusters to generate diversity in migrations. We understand that NC^{max} and w equal to 10 are enough to promote this diversity. However, it may be valuable to adjust these parameters in future experiments.

Table 1 lists the parameter settings of distributed models with 4 independent islands. For migratory frequency, FIX sets up migrations every 100 generations, while PROB defines a 5% chance of a migration occurring in each generation. The migratory policy of traditional migration strategies (FIX and PROB) is defined as *best-to-random*. Regarding the synchronization type, DDMS presents a method to trigger the migration. Therefore, it is more appropriate to use an asynchronous migration. In PROB, the asynchronous model is also used since the generation in which migration occurs is not known *a priori*. On the other hand, FIX is implemented by synchronizing the islands in the iteration whose migration is supposed to occur.

The algorithms were implemented in *C++/MPI* and compiled using GNU Compiler Collection (*GCC*) version 7.5.0.[3] The computational system was an Ubuntu 18.04 with 8 cores and 30 GB of memory.

Table 1
Parameter setting of distributed model for each strategy.

Parameter	Value
Number of Islands	4
DDMS	
Migratory topology	Pool
Migratory policy	Mean individual substitutes random
Synchronization	Asynchronous
FIX	
Migratory topology	Ring
Migratory frequency	100
Migratory policy	Best individual substitutes random
Synchronization	Synchronous
PROB	
Migratory topology	Ring
Migratory pate	0.05
Migratory policy	Best individual substitutes random
Synchronization	Asynchronous

Wilcoxon rank sum test is adopted to perform the statistical analysis. The significant level is set to 0.05, and the p-value is the significance factor. If DDMS wins over other strategies, its count of wins is incremented; if DDMS ties or losses, its count of ties or losses is incremented.

We are also interested in assessing whether the proposed migration strategy outperforms those more migration traditional approaches. Tables 2 and 3 show the count of wins, ties, and losses for DDMS when compared to Distributed EA with traditional migration strategies (FIX and PROB).[4] In these experiments, we define four scenarios, with $D = 10, 30, 50,$ and 100 variables.

The results reveal that the count of wins of DDMS is higher when the number of variables increases (D = 50 and 100), especially for hybrid (H) and composition (C) functions. Such a result indicates DDMS can better deal with higher dimension problems and complex landscapes that usually resemble real-world problems. Observe DDMS performs better than traditional migration strategies (FIX and PROB) in more

[3] The source codes and the supplementary files from the experiments are available for download at https://github.com/jeanto/migration-strategies.

[4] All strategies used L-SHADE as the optimizer.

Table 2
Counts of wins, losses and ties of DDMS *vs.* FIX according to significance of *Wilcoxon* rank-sum test.

DDMS	Type of function by dimension																			
	D = 10					D = 30					D = 50					D = 100				
	U	SM	H	C	Tot.	U	SM	H	C	Tot.	U	SM	H	C	Tot.	U	SM	H	C	Tot.
# wins	0	4	1	0	5	0	8	2	1	11	0	7	5	3	15	0	6	4	7	17
# losses	0	6	4	2	12	0	2	4	4	10	0	5	1	4	10	0	7	2	0	9
# ties	3	3	1	6	13	3	3	0	3	9	3	1	0	1	5	3	0	0	1	4

Table 3
Counts of wins, losses and ties of DDMS *vs.* PROB according to significance of *Wilcoxon* rank-sum test.

DDMS	Type of function by dimension																			
	D = 10					D = 30					D = 50					D = 100				
	U	SM	H	C	Tot.	U	SM	H	C	Tot.	U	SM	H	C	Tot.	U	SM	H	C	Tot.
# wins	0	5	3	1	9	0	8	1	2	11	0	7	5	3	15	0	6	5	8	19
# losses	0	5	2	2	9	0	2	5	4	11	0	5	1	3	9	0	6	0	0	6
# ties	3	3	1	5	12	3	3	0	2	8	3	1	0	2	6	3	1	1	0	5

Table 4
Mean ranks from *Friedman* test for each algorithm and dimension.

Algorithm	D = 10	D = 30	D = 50	D = 100	Avg.
DDMS	2.64 (3)	2.91 (1)	3.02 (1)	3.28 (1)	2.96
FIX	2.92 (1)	2.83 (3)	2.77 (2)	2.76 (2)	2.82
PROB	2.78 (2)	2.87 (2)	2.68 (3)	2.16 (3)	2.62

than half of the test problems, and the count of wins is larger than the count of losses in both scenarios.

Table 4 shows the *Friedman* test results. Considering the average ranks, we see DDMS performs significantly better than the others. Still, FIX proved to be more efficient than PROB. We can conclude that adaptive migration strategies can strongly influence the behavior of subpopulations and, consequently, impact the results obtained. Therefore, the moment of migration is the factor that can promote a significant positive impact on the performance of distributed evolutionary algorithms especially when the number of variables increases.

In Appendix A, the error values ($f(\mathbf{x}) - f(\mathbf{x}^*)$) are given in Tables 12, 13, 14, and 15 for 10, 30, 50, and 100 dimensions, respectively.

4.1. The effectiveness of the proposed strategy on migratory frequency

We choose some benchmark tests to demonstrate when migrations occur according to the type of function. The goal is to illustrate the approach that defines the proper moment of migration into DDMS. Fig. 4 shows the moment when migration happens on D = 50 in two functions (f_2, f_{12}, f_{17} and f_{29}) in each island. It shows that migrations in traditional strategies (FIX and PROB) are constant throughout evolution since they do not assess the loss of diversity on the islands. On the other hand, migrations in DDMS are more concentrated after half the run, when islands generally lose diversity. This mechanism helps to avoid unnecessary migrations at the beginning of the process.

We see that FIX is the strategy that gets the smallest number of migrations, while PROB migrates about 5 times more than FIX due to a migration rate of 0.05. Note that a higher number of migrations of the PROB does not imply better performance concerning the average error, as shown in Tables 2 and 3. These results suggest that, among traditional strategies, it is better to migrate less often and steadily across generations than simply to migrate more.

Figs. 4(b), 4(c), 4(e), 4(f), 4(h), 4(i), 4(k), and 4(l) show that the migrations in traditional strategies are constant throughout evolution since they do not assess the loss of diversity on the islands. On the other hand, migrations in DDMS are more concentrated after a third of the run, which is when islands generally lose diversity, as shown in the Figs. 4(a), 4(d), 4(g), and 4(j).

Depending on the function, it is common for islands to converge to the same regions of the search space. According DDMS, this fact tends to cause more migration due to the loss of diversity. In this sense, it seems clear that DDMS is the strategy that best suits the needs of the islands. This is because it seeks to maintain the historical backing of better individuals during migrations through the *TEDA*-Cloud. By clustering these individuals, it is possible to migrate an individual of good fitness and, at the same time, different from the population of the source island. However, as it was shown in the previous section, this does not mean better convergence in scenarios with fewer variables.

4.2. The effectiveness of the proposed strategy on exploration and exploitation

In this section, we are interested in further investigating the effectiveness of the proposed DDMS concerning exploration and exploitation.

Regarding exploration, we can observe its diversity preservation ability. Standard deviation is a frequently used statistic to assess the diversity of communities [45,46]. We compute the diversity according to Eq. (27) [45]:

$$\mu_{\sigma_g} = \frac{1}{\mathrm{D}} \sum_{i=1}^{\mathrm{D}} \sigma_i \tag{27}$$

in which D is the number of variables. Then, we generate a diversity indicator by computing the average of μ_σ in each generation g; besides, we plot this information on a graph that represents the diversity profiles.

To investigate the effectiveness of the method concerning exploitation, we can observe the error curves of the strategies through convergence profiles.

Figs. 5, 6, 7 and 8 show some convergence and diversity profiles with D = 50. The logarithmic values of the average errors are used for clarity.

Fig. 5(a) shows the convergence speed for f_2 function. Observe that all algorithms reach the minimum error. This result suggests that implementing distributed algorithms may be unnecessary in some scenarios, mainly in unimodal functions.

Fig. 5(b) shows that DDMS loses diversity faster. Figs. 6(b), 6(d), 7(b), 7(d), 8(b) and 8(d) demonstrate that this behavior is recurrent in the vast majority of the experiments performed. However, Figs. 6(a), 6(c), 7(a), 7(c), 8(a) and 8(c) show that this little diversity is "healthy", as the DDMS converges to a better final solution in most cases.

DDMS allows the islands to evolve independently. Migrations start only when the algorithm identifies the loss of diversity, as shown in Fig. 4. In this sense, DDMS seeks to suit the needs of the islands by

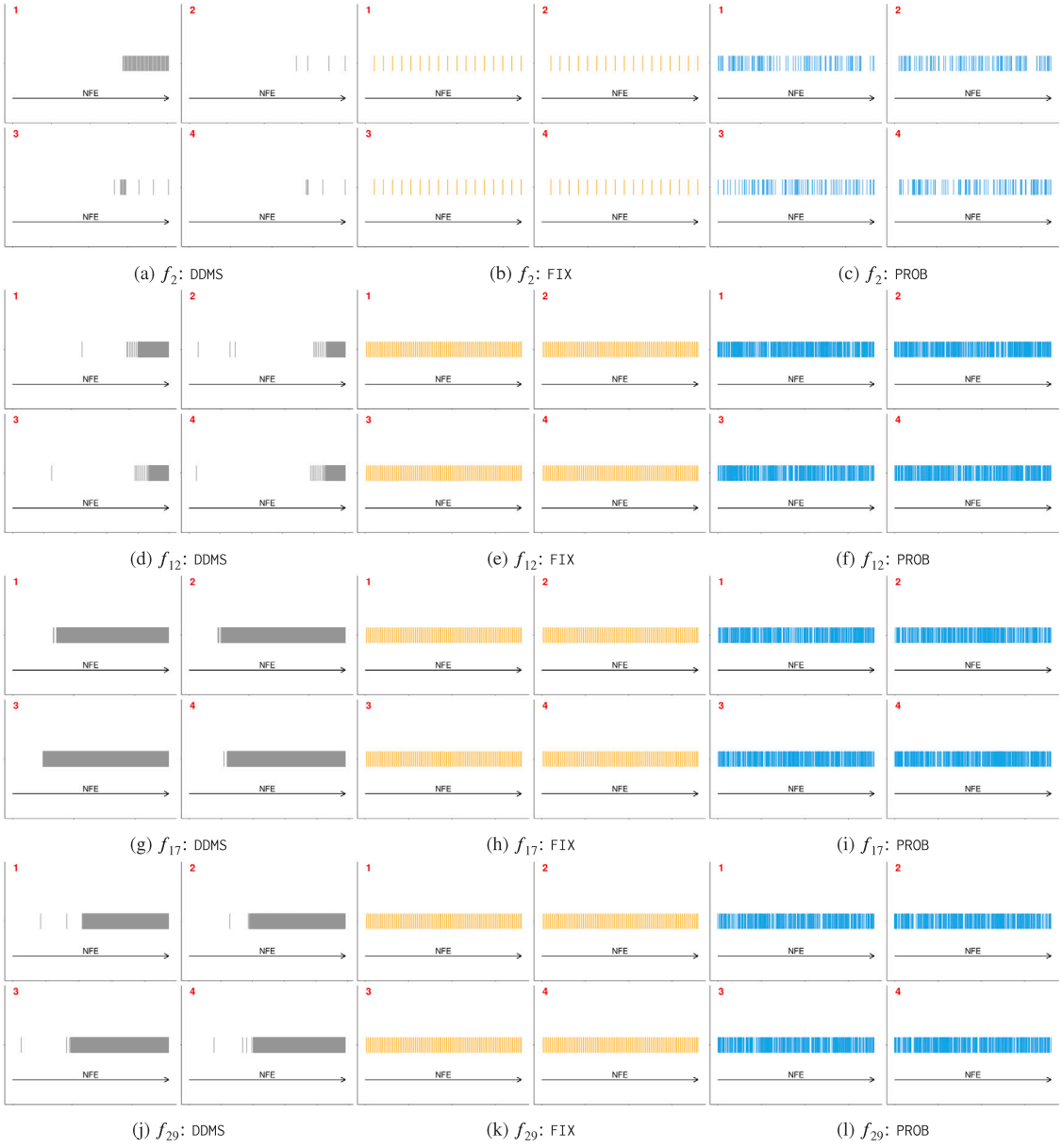


Fig. 4. Moment when migration happens in f_2, f_{12}, f_{17} and f_{29} functions for D = 50.

maintaining the historical backing of better individuals during migrations through the *TEDA*-Cloud. By clustering these individuals, it is possible to migrate an individual of reasonable fitness and, at the same time, different from the population of the target island. The effect of this strategy is to generate less diversity but with highly fit (healthy) individuals.

With deep insight, DDMS outperforms the other strategies for f_{13}, f_{17}, f_{20}, f_{29}, and f_{30} benchmark functions. In general, DDMS shows competitive convergence in multimodal and nonseparable (f_{13}), hybrid (f_{17}, f_{20}), and compositive benchmarks (f_{29}, f_{30}).

DDMS does not outperform the other strategies in some functions, for example, in f_{12} (Fig. 6(a)). It means DDMS can be advantageous to perform constant migrations for some functions, even if the overall diversity has not been lost.

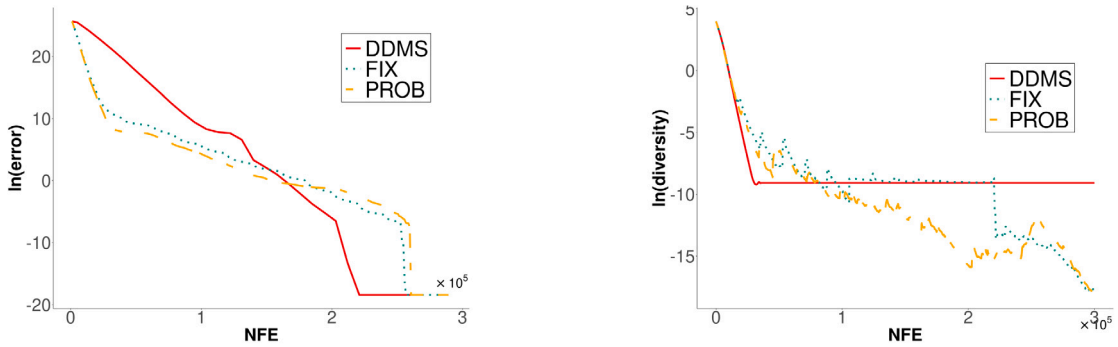
Figs. 9, 10, 11, and 12 show some convergence and diversity profiles with $D = 100$. The logarithmic values of the average errors are used for clarity.

Fig. 9(a) shows the convergence speed for f_2 function. In the same way for D = 50, all algorithms reach the minimum error. Again, this makes clear that unimodal functions do not require the implementation of distributed algorithms.

Figs. 9(b), 10(b), 10(d), 11(b), 11(d), 12(b) and 12(d) demonstrate that DDMS loses diversity faster than the other strategies since it tends not to perform migrations in the early stages of the evolution. These results are similar to those obtained in the scenario with $D = 50$. The reasons are the same as those discussed above. In summary, the proposed approach seeks to maintain a diversity capable of helping the population escape from local minimums and, at the same time, preserve the high quality of individuals during the migration stage.

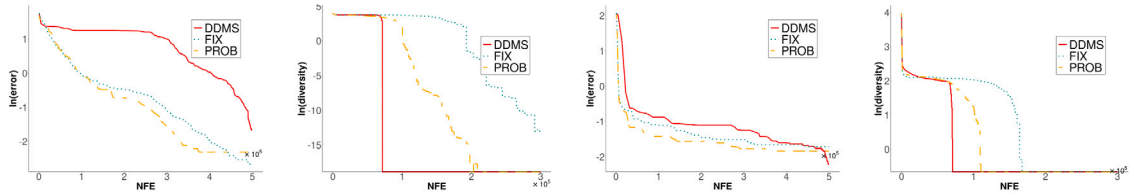
The convergence curves can be seen in Figs. 9(a), 10(a), 10(c), 11(a), 11(c), 12(a) and 12(c). Clearly, except for f_{12}, DDMS outperforms the other strategies in all the cases, just as it did in the scenario with D = 50.

From the experimental analysis, it is possible to validate our hypothesis on the proposed strategy: for all the problems under study, DDMS reaches competitive convergence compared to traditional algorithms, obtaining outstanding performance in hybrid and composite



(a) f_2: Convergence profile (b) f_2: Diversity profile

Fig. 5. Convergence and diversity profiles on D = 50 in Unimodal (U) functions.



(a) f_{12}: Convergence profile (b) f_{12}: Diversity profile (c) f_{13}: Convergence profile (d) f_{13}: Diversity profile

Fig. 6. Convergence and diversity profiles on D = 50 in Simple Multimodal (SM) functions.

Table 5
Counts of wins, losses and ties of DDMS *vs. SHADE* according to significance of *Wilcoxon* rank-sum test.

DDMS	Type of function by dimension									
	D = 50					D = 100				
	U	SM	H	C	Tot.	U	SM	H	C	Tot.
# wins	0	9	6	7	22	0	13	5	4	22
# losses	0	2	0	0	2	0	0	1	1	2
# ties	3	2	0	1	6	3	0	0	3	6

Table 6
Counts of wins, losses and ties of DDMS *vs. L-SHADE* according to significance of *Wilcoxon* rank-sum test.

DDMS	Type of function by dimension									
	D = 50					D = 100				
	U	SM	H	C	Tot.	U	SM	H	C	Tot.
# wins	0	9	6	7	22	0	12	5	4	21
# losses	0	4	0	0	4	0	1	1	1	3
# ties	3	0	0	1	4	3	0	0	3	6

problems with D = 100. Furthermore, DDMS strikes a reasonable balance between exploration and exploitation throughout the optimization process.

In summary, we can positively indicate that migration frequency and policy should be carefully analyzed when designing distributed evolutionary algorithms, as these factors significantly influence optimization performance.

5. DDMS *vs.* sequential EAs

Tables 5, 6, and 7 show the count of wins, ties, and losses for DDMS when compared to Sequential EAs (*SHADE*, *L-SHADE* and *SHADE-ILS*). The results show that DDMS is significantly better than Sequential EAs in more than two-thirds of the functions, and the count of wins is much larger than the count of losses in both scenarios (50D and 100D).

Table 7
Counts of wins, losses and ties of DDMS *vs. SHADE-ILS* according to significance of *Wilcoxon* rank-sum test.

DDMS	Type of function by dimension									
	D = 50					D = 100				
	U	SM	H	C	Tot.	U	SM	H	C	Tot.
# wins	0	8	2	5	15	0	12	5	4	21
# losses	0	3	2	1	6	0	1	1	1	3
# ties	3	1	3	2	9	3	0	0	3	6

Table 8
Mean ranks from *Friedman* test for each algorithm and dimension.

Algorithm	D = 50		D = 100		Avg.
DDMS	3.02	(1)	3.28	(1)	3.15
SHADE-ILS	2.63	(2)	1.80	(2)	2.22
L-SHADE	1.79	(3)	1.80	(3)	1.80
SHADE	1.66	(4)	1.77	(4)	1.72

Table 9
Complexity for the proposed algorithms.

Time complexity		
1	Assessing the loss of diversity	$O(3 \times NP + 2) \times D$
2	Get dimensions that should be diversified (Algorithm 1, line 13)	$O(D)$
3	*TEDA*-Cloud (Algorithm 1, line 18)	$O(D \times n \times NC)$

In general, distributed approaches tend to be more efficient than sequential ones, mainly because inter-island communication can help the population within the island to escape from local minimums.

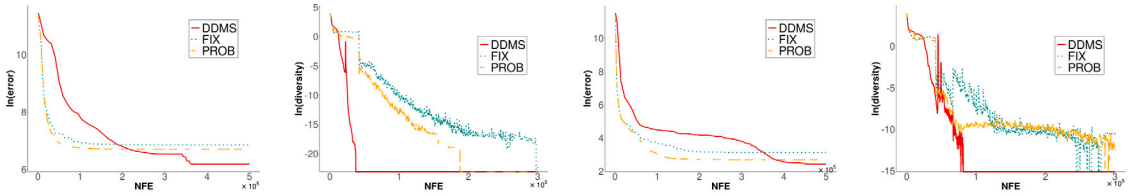
The strategies were also compared by *Friedman* test performed on medians of minimal function value errors concerning 50D and 100D scenarios. The null hypothesis of equivalent efficiency of the strategies was rejected for both dimensions with achieved significance level $p < 2.2 \times 10^{-16}$. The average ranks of the algorithms according to *Friedman* test are shown in Table 8. The algorithms are ordered concerning the

Table 10
Average error on D = 50: DDMS *vs.* Sequential EAs. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error and the *p*-value, respectively.

Fun.	DDMS	SHADE	L-SHADE	SHADE-ILS
f_1	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_2	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_3	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_4	**1.36e+01 (±1.10e+00)**(−)	3.04e+01 (±4.18e−02)(4.18e−03)	2.97e+01 (±3.11e−02)(2.49e−03)	2.19e+01 (±8.12e−02)(1.03e−03)
f_5	2.02e+01 (±3.09e−03)(−)	2.03e+01 (±2.46e−03)(2.18e−01)	**2.01e+01 (±9.56e−04)(1.65e−04)**	**2.01e+01 (±1.14e−03)(8.18e−05)**
f_6	**1.27e−05 (±9.58e−07)**(−)	1.29e+01 (±1.33e−07)(1.65e−04)	3.20e−01 (±3.83e−02)(1.65e−04)	1.28e+01 (±1.07e−01)(1.41e−07)
f_7	**0.00e+00 (±0.00e+00)**(−)	5.91e−03 (±9.21e−08)(3.18e−14)	5.42e−03 (±0.00e+00)(2.19e−14)	5.42e−03 (±0.00e+00)(2.19e−14)
f_8	1.04e−08 (±7.86e−10)(−)	**0.00e+00 (±0.00e+00)(9.92e−13)**	**0.00e+00 (±0.00e+00)(2.19e−14)**	**0.00e+00 (±0.00e+00)(1.98e−14)**
f_9	**1.42e+01 (±1.63e−01)**(−)	3.17e+01 (±3.37e−01)(0.00e+00)	3.11e+01 (±4.98e−01)(0.00e+00)	3.13e+01 (±2.18e−01)(0.00e+00)
f_{10}	2.85e−01 (±3.09e−03)(−)	5.67e−03 (±3.40e−01)(5.14e−03)	**1.86e−03 (±2.86e−01)(1.65e−04)**	1.91e−03 (±2.11e−01)(1.73e−04)
f_{11}	**2.68e+03 (±2.62e+01)**(−)	4.22e+03 (±2.13e+01)(7.31e−08)	3.51e+03 (±5.04e+01)(1.65e−04)	3.66e+03 (±1.98e+01)(8.21e−04)
f_{12}	**1.60e−01 (±1.13e−03)**(−)	**1.59e−01 (±1.74e−03)(7.91e−01)**	**1.58e−01 (±2.52e−03)(5.69e−01)**	**1.58e−01 (±1.99e−03)(4.78e−01)**
f_{13}	**1.17e−01 (±1.17e−03)**(−)	2.92e−01 (±1.76e−03)(0.00e+00)	2.90e−01 (±2.07e−03)(0.00e+00)	2.91e−01 (±2.06e−03)(0.00e+00)
f_{14}	**2.59e−01 (±4.76e−04)**(−)	3.58e−01 (±6.79e−03)(0.00e+00)	3.52e−01 (±2.04e−03)(0.00e+00)	3.19e−01 (±1.75e−03)(0.00e+00)
f_{15}	**4.69e+00 (±2.45e−02)**(−)	6.66e+00 (±4.59e−02)(0.00e+00)	6.58e+00 (±4.88e−02)(0.00e+00)	5.40e+00 (±3.58e−02)(0.00e+00)
f_{16}	**1.57e+01 (±2.44e−02)**(−)	1.74e+01 (±2.49e−02)(1.22e−06)	1.74e+01 (±8.90e−02)(1.27e−06)	1.69e+01 (±3.86e−02)(7.35e−05)
f_{17}	**3.48e+02 (±8.77e+00)**(−)	1.45e+03 (±2.07e+01)(0.00e+00)	1.44e+03 (±2.20e+01)(0.00e+00)	9.92e+02 (±1.86e+01)(0.00e+00)
f_{18}	**9.00e+01 (±4.33e−01)**(−)	1.62e+02 (±2.23e+00)(0.00e+00)	1.60e+02 (±9.66e−01)(0.00e+00)	**9.01e+01 (±5.66e−01)(2.17e−01)**
f_{19}	8.57e+00 (±9.36e−02)(−)	1.03e+01 (±1.55e−01)(1.16e−04)	1.01e+01 (±4.23e−02)(1.65e−04)	**7.99e+00 (±1.29e−01)(1.24e−02)**
f_{20}	**8.06e+00 (±1.18e−01)**(−)	5.19e+01 (±1.26e+00)(0.00e+00)	5.17e+01 (±1.96e−01)(0.00e+00)	**7.93e+00 (±1.36e−01)(1.87e−01)**
f_{21}	**3.57e+02 (±4.74e+00)**(−)	1.49e+03 (±2.01e+02)(0.00e+00)	1.51e+03 (±8.62e+00)(0.00e+00)	**3.61e+02 (±6.21e+00)(3.72e−01)**
f_{22}	1.36e+02 (±1.84e+00)(−)	3.49e+02 (±5.33e+00)(0.00e+00)	3.52e+02 (±6.44e+00)(0.00e+00)	**1.15e+02 (±6.42e+00)(2.45e−03)**
f_{23}	**1.24e+02 (±0.00e+00)**(−)	**1.24e+02 (±0.00e+00)(1.00e+00)**	**1.24e+02 (±0.00e+00)(1.00e+00)**	**1.24e+02 (±0.00e+00)(1.00e+00)**
f_{24}	**2.70e+02 (±8.38e−02)**(−)	2.80e+02 (±1.61e−01)(0.00e+00)	2.80e+02 (±1.35e−01)(0.00e+00)	2.75e+02 (±3.62e−02)(0.00e+00)
f_{25}	**2.00e+02 (±2.19e−04)**(−)	2.00e+02 (±1.16e−03)(8.54e−09)	2.00e+02 (±1.37e−03)(1.30e−09)	2.00e+02 (±1.04e−03)(4.66e−08)
f_{26}	**1.00e+02 (±6.96e−04)**(−)	1.00e+02 (±2.99e−03)(9.26e−12)	1.00e+02 (±1.52e−03)(7.71e−13)	1.00e+02 (±1.29e−03)(6.62e−12)
f_{27}	**4.00e+02 (±3.21e−05)**(−)	4.00e+02 (±1.40e−04)(6.64e−05)	4.00e+02 (±4.79e−05)(1.00e−04)	4.00e+02 (±2.31e−05)(1.23e−02)
f_{28}	5.73e+02 (±4.29e−01)(−)	6.06e+02 (±1.29e+00)(9.39e−08)	6.06e+02 (±1.02e+00)(1.11e−07)	**5.58e+02 (±1.28e+00)(9.14e−03)**
f_{29}	**7.48e+02 (±3.58e+00)**(−)	8.86e+02 (±8.10e+00)(0.00e+00)	8.94e+02 (±6.27e+00)(0.00e+00)	7.94e+02 (±2.42e+00)(0.00e+00)
f_{30}	**6.23e+02 (±1.44e+00)**(−)	9.62e+02 (±1.16e+01)(0.00e+00)	9.60e+02 (±4.77e+00)(0.00e+00)	6.45e+02 (±3.80e+00)(0.00e+00)

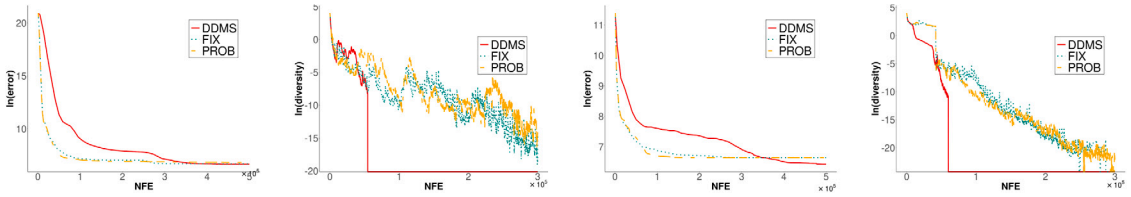
Table 11
Average error on D = 100: DDMS *vs.* Sequential EAs. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error and the *p*-value, respectively.

Fun.	DDMS	SHADE	L-SHADE	SHADE-ILS
f_1	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**	7.93e−06 (±0.00e+00)(1.39e−14)
f_2	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**	0.00e+00 (±0.00e+00)(1.00e+00)
f_3	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**	0.00e+00 (±0.00e+00)(1.00e+00)
f_4	**9.34e+01 (±2.50e+00)**(−)	1.09e+02 (±2.65e+00)(6.95e−04)	1.09e+02 (±1.28e+00)(6.69e−04)	1.06e+02 (±2.94e+00)(9.11e−03)
f_5	**2.04e+01 (±1.33e−02)**(−)	2.05e+01 (±4.67e−03)(4.25e−04)	2.05e+01 (±1.85e−04)(3.85e−04)	2.05e+01 (±4.25e−03)(4.01e−04)
f_6	**7.01e+00 (±6.11e−02)**(−)	3.90e+01 (±1.53e−01)(1.01e−04)	3.91e+01 (±2.77e−01)(1.29e−04)	3.87e+01 (±1.89e−01)(9.92e−03)
f_7	**0.00e+00 (±0.00e+00)**(−)	5.48e−03 (±3.80e−04)(0.00e+00)	5.29e−03 (±2.26e−04)(0.00e+00)	5.25e−03 (±3.59e−04)(0.00e+00)
f_8	2.86e−01 (±1.33e−02)(−)	2.97e−08 (±5.86e−09)(3.57e−04)	2.59e−08 (±0.00e+00)(6.32e−05)	**2.50e−08 (±5.38e−09)(3.36e−06)**
f_9	**4.08e+01 (±4.02e−01)**(−)	9.43e+01 (±4.86e−01)(2.01e−04)	9.43e+01 (±7.49e−01)(1.29e−04)	9.40e+01 (±4.42e−01)(7.65e−03)
f_{10}	**5.52e+01 (±2.94e−01)**(−)	7.78e−01 (±3.66e−02)(8.14e−03)	7.92e−01 (±9.44e−04)(1.29e−04)	7.68e−01 (±3.15e−02)(5.13e−03)
f_{11}	**9.62e+03 (±2.30e+01)**(−)	1.14e+04 (±1.62e+01)(7.12e−07)	1.14e+04 (±2.96e+01)(7.47e−07)	1.13e+04 (±1.43e+01)(6.34e−07)
f_{12}	**3.33e−01 (±2.06e−03)**(−)	3.70e−01 (±1.68e−03)(1.76e−04)	3.70e−01 (±1.48e−03)(1.29e−04)	3.69e−01 (±1.16e−03)(9.33e−03)
f_{13}	**2.11e−01 (±5.07e−04)**(−)	3.48e−01 (±2.01e−03)(0.00e+00)	3.47e−01 (±2.45e−03)(0.00e+00)	3.45e−01 (±2.25e−03)(0.00e+00)
f_{14}	**1.05e−01 (±6.07e−04)**(−)	1.31e−01 (±4.84e−04)(0.00e+00)	1.31e−01 (±6.11e−04)(0.00e+00)	1.31e−01 (±4.45e−04)(0.00e+00)
f_{15}	**1.23e+01 (±1.61e−01)**(−)	2.69e+01 (±1.99e−01)(0.00e+00)	2.70e+01 (±2.65e−01)(0.00e+00)	2.68e+01 (±1.98e−01)(0.00e+00)
f_{16}	**3.76e+01 (±4.11e−02)**(−)	3.94e+01 (±2.21e−02)(3.86e−05)	3.94e+01 (±5.01e−02)(7.33e−05)	3.94e+01 (±2.03e−02)(5.34e−05)
f_{17}	**2.23e+03 (±2.01e+01)**(−)	3.87e+03 (±3.81e+01)(0.00e+00)	3.86e+03 (±2.95e+01)(0.00e+00)	3.84e+03 (±3.64e+01)(0.00e+00)
f_{18}	**2.15e+02 (±6.50e−01)**(−)	4.24e+02 (±1.16e+01)(0.00e+00)	4.06e+02 (±2.07e+00)(0.00e+00)	4.10e+02 (±1.39e+01)(0.00e+00)
f_{19}	1.04e+02 (±2.61e−01)(−)	**1.01e+02 (±1.09e+00)(5.23e−03)**	**1.01e+02 (±1.16e+00)(4.16e−03)**	**1.01e+02 (±1.02e+00)(6.43e−03)**
f_{20}	**3.42e+01 (±2.54e−01)**(−)	1.56e+02 (±1.54e+00)(0.00e+00)	1.54e+02 (±2.61e+00)(0.00e+00)	1.53e+02 (±1.45e+00)(0.00e+00)
f_{21}	**6.60e+02 (±9.26e+00)**(−)	3.57e+03 (±5.51e+02)(0.00e+00)	3.30e+03 (±3.46e+01)(0.00e+00)	3.35e+03 (±4.58e+02)(0.00e+00)
f_{22}	**8.88e+02 (±1.22e+01)**(−)	1.27e+03 (±9.70e+00)(9.45e−15)	1.27e+03 (±9.57e+00)(8.10e−15)	1.27e+03 (±8.81e+00)(7.45e−15)
f_{23}	1.23e+02 (±0.00e+00)(−)	**1.22e+02 (±0.00e+00)(2.23e−03)**	**1.22e+02 (±0.00e+00)(2.23e−03)**	**1.22e+02 (±0.00e+00)(2.23e−03)**
f_{24}	**3.71e+02 (±1.02e−01)**(−)	4.04e+02 (±2.41e−01)(8.45e−06)	4.04e+02 (±3.36e−01)(9.44e−06)	4.04e+02 (±2.32e−01)(7.34e−06)
f_{25}	**2.01e+02 (±8.84e−04)**(−)	**2.01e+02 (±2.52e−03)(1.00e+00)**	**2.01e+02 (±1.17e−03)(1.00e+00)**	**2.01e+02 (±2.46e−03)(1.00e+00)**
f_{26}	**1.00e+02 (±1.32e−03)**(−)	**1.00e+02 (±2.24e−03)(1.00e+00)**	**1.00e+02 (±2.99e−03)(1.00e+00)**	**1.00e+02 (±1.96e−03)(1.00e+00)**
f_{27}	**4.00e+02 (±3.75e−05)**(−)	**4.00e+02 (±1.31e−04)(1.00e+00)**	**4.00e+02 (±1.93e−04)(1.00e+00)**	**4.00e+02 (±1.41e−04)(1.00e+00)**
f_{28}	**8.31e+02 (±1.85e+00)**(−)	8.71e+02 (±4.99e+00)(3.23e−04)	8.64e+02 (±6.23e+00)(1.29e−04)	8.65e+02 (±3.18e+00)(1.56e−04)
f_{29}	**7.13e+02 (±1.36e+00)**(−)	9.04e+02 (±8.55e+00)(3.78e−15)	8.96e+02 (±7.81e+00)(1.33e−15)	8.90e+02 (±8.87e+00)(8.56e−14)
f_{30}	**2.15e+03 (±1.26e+01)**(−)	2.88e+03 (±2.12e+01)(6.43e−05)	2.89e+03 (±2.31e+01)(9.66e−05)	2.87e+03 (±2.04e+01)(4.67e−05)



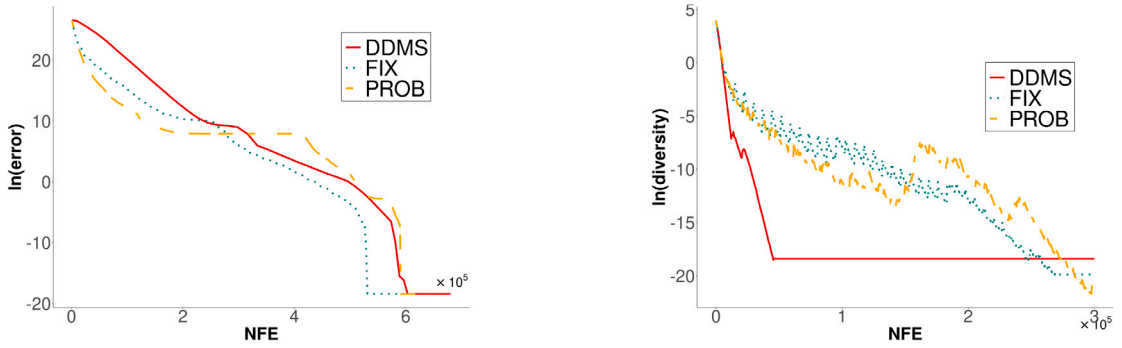
(a) f_{17}: Convergence profile (b) f_{17}: Diversity profile (c) f_{20}: Convergence profile (d) f_{20}: Diversity profile

Fig. 7. Convergence and diversity profiles on D = 50 in Hybrid (H) functions.



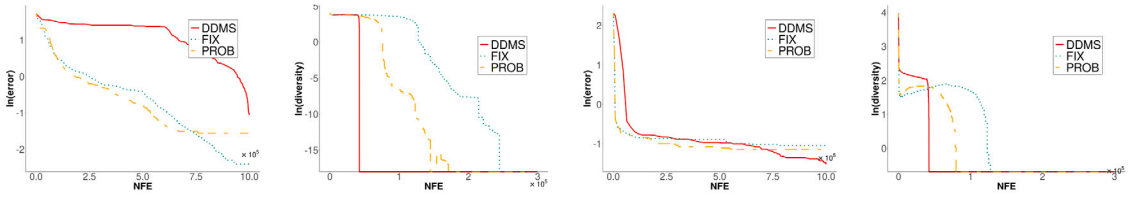
(a) f_{29}: Convergence profile (b) f_{29}: Diversity profile (c) f_{30}: Convergence profile (d) f_{30}: Diversity profile

Fig. 8. Convergence and diversity profiles on D = 50 in Composition (C) functions.



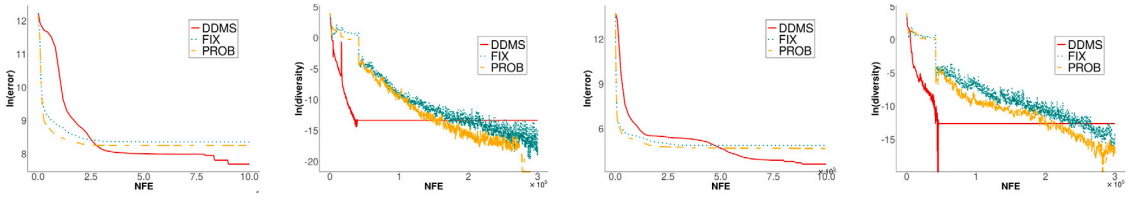
(a) f_2: Convergence profile (b) f_2: Diversity profile

Fig. 9. Convergence and diversity profiles on D = 100 in Unimodal (U) functions.



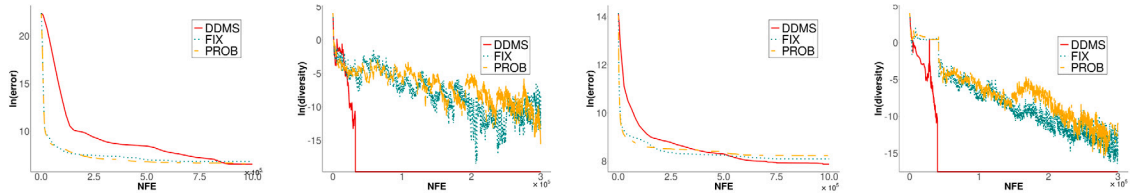
(a) f_{12}: Convergence profile (b) f_{12}: Diversity profile (c) f_{13}: Convergence profile (d) f_{13}: Diversity profile

Fig. 10. Convergence and diversity on D = 100 in Simple Multimodal (SM) functions.



(a) f_{17}: Convergence profile (b) f_{17}: Diversity profile (c) f_{20}: Convergence profile (d) f_{20}: Diversity profile

Fig. 11. Convergence and diversity profiles on D = 100 in Hybrid (H) functions.



(a) f_{29}: Convergence profile (b) f_{29}: Diversity profile (c) f_{30}: Convergence profile (d) f_{30}: Diversity profile

Fig. 12. Convergence and diversity profiles on D = 100 in Composition (C) functions.

average mean ranks overall dimensions. The ranks of the algorithms for each dimension level are shown in parentheses after the corresponding mean ranks.

Considering these average ranks, we can see DDMS performs significantly better than the others. It is also possible to observe DDMS performs very well when the number of variables increases. Still, *SHADE-ILS* was the best among the sequential methods.

The error values ($f(\mathbf{x}) - f(\mathbf{x}^*)$) are given in Tables 10 and 11 for 50D and 100D, respectively. The numbers in parentheses represent the standard error and the p-value of each algorithm, respectively. We added a bold highlight to the strategies that obtained the lowest average error for each function.

From results shown in Table 10, we can find that DDMS reaches the first position in 24 functions. Besides, it wins on 19 functions over all other algorithms. Table 11 shows the corresponding results on D = 100. For the average performance, it reaches the first position in 27 functions and wins on 21 benchmarks over all other algorithms.

Putting the type of test function into perspective, none of the strategies had difficulty in optimizing the unimodal instances (f_1–f_3). In other functions (f_4–f_{30}), DDMS had good performance mainly when the number of variables increased (D = 100).

Considering *SHADE-ILS* was the best one among sequential methods, we can assume that DDMS tends to improve the performance of EAs since it was able to overcome *SHADE-ILS* in most functions even running *L-SHADE* on the islands. Appendix B compare DDMS against some recent sequential EAs, namely LSHADE-EpSin [47], FLDE [48], MSHCSA [49], and ADECSA [50].

6. Computational complexity

Time complexity of the algorithms can be analyzed considering all the steps separately, as summarized in Table 9. DDMS takes $O(3 \times D \times \text{NP})$ to generate the trial vector and compute $\boldsymbol{\mu}$ and σ vectors for each dimension j. For computing Γ^g and $\boldsymbol{\beta}^g$, the computational complexity is linear in D ($O(D)$). Therefore, the first phase *"assessing the loss of diversity"* has complexity of $O(3 \times \text{NP} + 2) \times D$. We also added the second phase to check outlier dimensions, which is linear in D ($O(D)$). Thus, DDMS has complexity of $O(3 \times \text{NP} + 3) \times D$ on the island.

In the phase of clustering on the pool, DDMS is dimensionally dependent during the calculation of variance and eccentricity. Given that n is the number of samples to be clustered and NC is the number of *data clouds*, DDMS has a time complexity of $O(D \times n \times \text{NC})$, in which n is always equal to 1 (best individual) and the largest value of NC is equal to 10. Thus, DDMS has a time complexity of $O(D \times 10)$ to group an individual in the worst case, that is, when a sample belongs to all *data clouds*.

7. Conclusions

Some studies recognize the importance of defining the appropriate moment to perform migration in distributed evolutionary algorithms. However, most of them use at fixed intervals or with a given probability. Another issue involves the definition of how to migrate an individual such that it promotes a healthy diversity on the target island. Most studies focus on migrating the best or a random individual.

In this work, we presented rules to assess the loss of diversity in each dimension to obtain a trigger to perform migrations. Also, we proposed a strategy that aims to balance exploitation and exploration. Instead of creating a topology with direct communication between neighboring islands, migrations occur between the islands and a pool. After receiving the best individual from a given island, this pool clusters it in a cloud along with similar individuals using the *TEDA*-Cloud framework. After clustering, an individual sufficiently different from the target island is extracted from one of the clouds and sent to the target island. This approach seeks to maintain a recent history of good-fitness individuals while generating diversity by sending individuals different from the target island population.

We called this approach a diversity-driven migration strategy (DDMS). It was compared against usual strategies that consider periodic migrations that send the best individual and replace a random individual on the target island. The experimental comparison brings promising results.

Regarding diversity, DDMS was the profitable approach in most functions and scenarios. On convergence, DDMS is particularly effective when dealing with hybrid and composition functions when D = 100. When solving unimodal and simple multimodal problems, the benefit of employing our strategy depends on the test function.

The results demonstrate that a diversity-driven migration strategy can reduce unnecessary and unproductive migrations. Particularly, this approach is suitable for dealing with challenging landscapes, which often refer to complex real-world problems, and it also points to promising results in high-dimensional and large-scale problems.

In future work, we aim to extend this study by including large-scale problems. The purpose is to evaluate the scalability and computational time of the proposed approach. Besides, we hope to make a sensitivity analysis concerning the parameters of the *TEDA*-Cloud algorithm.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors would like to thank all the reviewers for their constructive comments. This work was supported by Brazilian agencies FAPEMIG (Research Support Foundation of the State of Minas Gerais), CNPq (The National Council for Scientific and Technological Development), and CAPES (Coordination for the Improvement of Higher Education Personnel).

Table 12
Average error on D = 10. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error and the *p*-value, respectively.

Fun.	DDMS	FIX	PROB
f_1	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_2	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_3	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_4	**2.39e−12 (±7.07e−01)**(–)	2.41e−10 (±5.87e−01)(1.17e−13)	2.33e−09 (±4.06e−01)(4.15e−15)
f_5	**2.00e+01 (±9.43e−04)**(–)	**2.00e+01 (±8.75e−03)(1.00e+00)**	**2.00e+01 (±9.73e−04)(1.00e+00)**
f_6	2.47e−11 (±5.38e−08)(–)	**0.00e−00 (±7.15e−10)(8.54e−09)**	6.01e−11 (±1.53e−11)(8.87e−09)
f_7	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_8	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_9	6.36e−09 (±6.13e−01)(–)	2.80e−02 (±8.84e−02)(9.93e−15)	**3.43e−09 (±5.61e−01)(6.55e−03)**
f_{10}	**6.25e−02 (±9.64e−04)**(–)	6.33e−02 (±7.46e−02)(1.41e−11)	1.25e−01 (±1.45e−03)(8.86e−06)
f_{11}	1.37e+01 (±6.24e+00)(–)	1.03e+01 (±8.23e+02)(1.95e−02)	**9.99e+00 (±1.93e+01)(5.45e−04)**
f_{12}	**2.34e−02 (±1.32e−02)**(–)	3.50e−02 (±1.35e−03)(8.21e−04)	2.84e−02 (±4.26e−03)(4.68e−03)
f_{13}	1.57e−02 (±1.77e−03)(–)	**8.41e−03 (±1.57e−03)(3.87e−03)**	1.43e−02 (±7.68e−03)(8.57e−02)
f_{14}	4.06e−02 (±7.68e−04)(–)	**3.87e−02 (±5.23e−03)(2.43e−09)**	4.39e−02 (±4.78e−03)(5.38e−08)
f_{15}	3.47e−01 (±4.45e−02)(–)	2.89e−01 (±7.24e−01)(6.13e−04)	**2.78e−01 (±6.32e−02)(9.51e−04)**
f_{16}	2.25e−01 (±4.34e−02)(–)	2.13e−01 (±6.15e−02)(9.12e−11)	**1.75e−01 (±4.60e−02)(2.11e−12)**
f_{17}	6.98e−09 (±7.73e+00)(–)	**1.93e−11 (±3.13e+02)(1.31e−13)**	2.31e−10 (±2.54e+01)(6.12e−10)
f_{18}	3.70e−03 (±3.21e−01)(–)	8.60e−04 (±2.31e−01)(8.12e−05)	**4.78e−04 (±9.66e−02)(9.39e−06)**
f_{19}	1.16e+00 (±3.61e−02)(–)	**1.13e+00 (±4.37e−01)(4.17e−06)**	1.19e+00 (±6.24e−02)(2.34e−07)
f_{20}	**5.00e−01 (±1.86e−01)**(–)	**5.00e−01 (±1.66e−01)(1.00e+00)**	**5.00e−01 (±3.31e−01)(1.00e+00)**
f_{21}	1.49e−04 (±7.34e−01)(–)	9.14e−05 (±7.65e−01)(2.30e−04)	**2.85e−05 (±4.23e−01)(3.82e−06)**
f_{22}	**6.76e+00 (±8.43e+00)**(–)	7.29e+00 (±2.53e+00)(3.32e−05)	1.81e+01 (±4.82e+00)(3.54e−08)
f_{23}	**1.60e+02 (±0.00e+00)**(–)	**1.60e+02 (±0.00e+00)(1.00e+00)**	**1.60e+02 (±0.00e+00)(1.00e+00)**
f_{24}	1.05e+02 (±3.82e−02)(–)	**1.00e+02 (±9.41e−00)(3.54e−03)**	**1.00e+02 (±5.63e−01)(3.36e−03)**
f_{25}	**1.00e+02 (±1.97e−04)**(–)	**1.00e+02 (±0.00e+00)(1.00e+00)**	**1.00e+02 (±9.84e−05)(1.00e+00)**
f_{26}	**1.00e+02 (±6.76e−04)**(–)	**1.00e+02 (±5.45e−04)(1.00e+00)**	**1.00e+02 (±4.74e−03)(1.00e+00)**
f_{27}	6.84e−01 (±2.51e−05)(–)	**6.12e−01 (±9.34e−06)(2.45e−04)**	7.89e−01 (±8.43e−05)(1.56e−04)
f_{28}	**3.02e+02 (±2.49e−01)**(–)	**3.02e+02 (±3.51e−02)(1.00e+00)**	**3.02e+02 (±1.94e−00)(1.00e+00)**
f_{29}	**2.01e+02 (±5.48e+00)**(–)	**2.01e+02 (±1.93e+00)(1.00e+00)**	**2.01e+02 (±1.13e+00)(1.00e+00)**
f_{30}	2.20e+02 (±4.23e+00)(–)	2.20e+02 (±3.84e+00)(5.32e−04)	**2.02e+02 (±4.32e+00)(3.32e−08)**

Appendix A. DDMS *vs.* traditional migration strategies

In this appendix, we listed the error values ($f(\mathbf{x}) - f(\mathbf{x}^*)$) of the proposed migration strategy (DDMS) compared to the migration traditional approaches (FIX and PROB).

Tables 12, 13, 14, and 15 show the error values for 10, 30, 50, and 100 dimensions, respectively. The numbers in parentheses represent the standard error and the *p*-value of each algorithm. We added a bold highlight to the strategies that obtained the lowest average error for each function.

From the results shown in Table 12, we can find that DDMS reaches the first position in 16 functions. Besides, it wins on 3 functions over all other algorithms. Table 13 shows the corresponding results on D = 30. For the average performance, DDMS reaches the first position in 17 functions and wins on 9 benchmarks over all other algorithms.

From the results shown in Table 14, DDMS reaches the first position in 19 functions and wins on 15 functions over all other algorithms. Table 15 shows the results on D = 100. DDMS reaches the first position in 22 functions and wins on 17 benchmarks over all other algorithms.

Putting the type of function with a greater number of variables into perspective, none of the strategies had difficulty in optimizing the unimodal functions (f_1–f_3). In simple multimodal functions (f_4–f_{16}), we can see a balance between DDMS and traditional strategies. In hybrid and composition functions (f_{17}–f_{30}), DDMS had good performance. Especially in composition functions for D = 100, DDMS performed better than the other ones. In fact, when looking into Tables 14 and 15, we can see that DDMS is more favorable for solving problems with hybrid and composition functions in the scenarios of greater dimensionality.

Appendix B. DDMS *vs.* recent sequential EAs

We have also compared DDMS against some recent sequential EAs, namely LSHADE-EpSin [47], FLDE [48], MSHCSA [49], and ADECSA [50].

LSHADE-EpSin is an algorithm which uses a ensemble sinusoidal approach to automatically adapt the values of the scaling factor of the DE [47]. The approach consists of a mixture of two sinusoidal formulas: a non-adaptive sinusoidal decreasing adjustment and an adaptive history-based sinusoidal increasing adjustment. The idea is to find an effective balance between the exploitation of the already found best solutions, and the exploration of non-visited regions. The work demonstrated the efficiency and robustness of the LSHADE-EpSin to obtain better results when compared to *L-SHADE* algorithm and other state-of-the-art algorithms.

FLDE is an DE-based algorithm which applies an adaptive mutation strategy based on fitness landscape [48]. First, the algorithm analyzes the fitness landscape features of each benchmark function. Then, the relationship between three mutation strategies and fitness landscape features is trained by random forest (RF) offline. Finally, the trained RF is used to predict which mutation strategy should be utilized to perform mutation operator for each problem during the evolutionary process. The authors showed that FLDE was highly competitive when compared with others DE algorithms.

MSHCSA is an algorithm that proposes a modified combinatorial recombination to bring diversity to the population and avoid premature convergence [49]. A success-history based adaptive mutation strategy is introduced to form a selection algorithm that improves the search ability. The mutation operator is also modified and analyzed through experimental comparison. To cope with the stagnation, the gene knockout strategy is proposed. The experimental results showed that MSHCSA is quite competitive when compared with some state-of-the-art algorithms.

The recent work presented in [50] propose an improved clonal selection algorithm called an adaptive clonal selection algorithm with multiple differential evolution strategies (ADECSA). The algorithm has three features: (1) an adaptive mutation strategy pool based on its historical records of success is introduced to guide the immune response process effectively; (2) an adaptive population resizing method is

Table 13
Average error on D = 30. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error and the *p*-value, respectively.

Fun.	DDMS	FIX	PROB
f_1	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_2	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_3	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_4	2.12e+00 (±9.72e−01)(−)	**2.03e+00 (±6.74e−01)(7.76e−14)**	1.96e+01 (±6.44e−01)(5.22e−14)
f_5	**2.00e+01 (±4.33e−04)**(−)	**2.00e+01 (±7.45e−03)(1.00e+00)**	**2.00e+01 (±4.40e−04)(1.00e+00)**
f_6	**3.34e+00 (±3.38e−08)**(−)	8.98e+00 (±5.42e−09)(4.23e−08)	7.48e+00 (±2.31e−10)(1.71e−08)
f_7	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_8	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_9	**2.05e+00 (±7.32e−01)**(−)	9.08e+00 (±6.46e−02)(8.37e−14)	3.02e+00 (±4.14e−01)(5.32e−03)
f_{10}	**1.06e−01 (±6.42e−04)**(−)	1.59e+00 (±4.26e−02)(1.34e−06)	1.27e+00 (±5.64e−03)(8.75e−03)
f_{11}	**3.73e+02 (±2.34e+00)**(−)	5.71e+02 (±8.30e+02)(2.51e−02)	4.63e+02 (±1.31e+02)(1.54e−02)
f_{12}	**6.37e−01 (±2.48e−02)**(−)	6.55e−01 (±9.56e−03)(4.14e−03)	6.43e−01 (±5.36e−03)(9.83e−02)
f_{13}	2.87e−01 (±3.12e−03)(−)	**2.50e−01 (±6.71e−03)(1.71e−03)**	2.77e−01 (±6.18e−03)(9.01e−02)
f_{14}	**2.16e−01 (±8.34e−04)**(−)	2.31e−01 (±2.13e−03)(1.30e−08)	2.23e−01 (±8.03e−03)(3.38e−06)
f_{15}	**8.74e+00 (±1.25e−02)**(−)	1.13e+01 (±4.41e−01)(3.32e−04)	1.18e+01 (±5.21e−02)(8.13e−04)
f_{16}	1.16e+01 (±7.54e−02)(−)	1.18e+01 (±4.21e−02)(1.23e−02)	**1.15e+01 (±6.20e−02)(1.11e−02)**
f_{17}	1.40e+01 (±3.39e+00)(−)	1.51e+01 (±6.35e+02)(4.29e−03)	**1.26e+01 (±4.42e+01)(2.27e−03)**
f_{18}	5.09e+01 (±1.17e−01)(−)	**5.01e+01 (±4.31e−01)(4.22e−02)**	5.02e+01 (±6.36e−02)(3.19e−02)
f_{19}	7.21e+00 (±1.17e−02)(−)	6.97e+00 (±3.37e−01)(7.43e−04)	**6.07e+00 (±5.44e−02)(3.62e−05)**
f_{20}	8.90e−01 (±1.62e−01)(−)	8.70e−01 (±1.16e−01)(2.02e−04)	**5.91e−01 (±4.13e−01)(6.03e−10)**
f_{21}	3.72e+02 (±3.64e−01)(−)	5.35e+02 (±5.53e−01)(3.04e−07)	**3.38e+02 (±2.43e−01)(2.32e−03)**
f_{22}	3.44e+01 (±3.38e+00)(−)	**3.22e+01 (±1.34e+00)(7.27e−05)**	3.85e+01 (±8.332e+00)(9.54e−05)
f_{23}	**1.00e+02 (±0.00e+00)**(−)	**1.00e+02 (±0.00e+00)(1.00e+00)**	**1.00e+02 (±0.00e+00)(1.00e+00)**
f_{24}	2.01e+02 (±2.21e−02)(−)	**2.00e+02 (±8.11e−00)(4.12e−03)**	**2.00e+02 (±7.33e−01)(4.61e−03)**
f_{25}	2.00e+02 (±2.71e−04)(−)	**2.00e+02 (±0.00e+00)(3.91e−02)**	**2.00e+02 (±4.62e−05)(2.91e−02)**
f_{26}	1.00e+02 (±6.76e−04)(−)	**1.00e+02 (±5.45e−04)(4.00e−04)**	**1.00e+02 (±4.74e−03)(1.25e−01)**
f_{27}	4.00e+02 (±1.99e−05)(−)	**4.00e+02 (±5.73e−05)(1.00e−04)**	**4.00e+02 (±7.37e−05)(1.00e−04)**
f_{28}	**3.02e+02 (±3.91e−01)**(−)	**3.02e+02 (±2.11e−02)(1.00e+00)**	**3.02e+02 (±2.04e−00)(1.00e+00)**
f_{29}	**2.12e+02 (±4.18e+00)**(−)	2.15e+02 (±1.36e+00)(3.31e−03)	2.14e+02 (±1.33e+00)(8.01e−02)
f_{30}	**2.95e+02 (±6.34e+00)**(−)	4.46e+02 (±2.42e+00)(3.22e−07)	3.28e+02 (±3.22e+00)(6.25e−05)

Table 14
Average error on D = 50. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error and the *p*-value, respectively.

Fun.	DDMS	FIX	PROB
f_1	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_2	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_3	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_4	1.36e+01 (±1.10e+00)(−)	**1.39e−01 (±1.58e−01)(2.11e−15)**	7.08e−01 (±4.40e−02)(2.41e−05)
f_5	2.02e+01 (±3.09e−03)(−)	**2.00e+01 (±5.18e−03)(0.00e+00)**	**2.00e+01 (±3.97e−03)(0.00e+00)**
f_6	1.27e−05 (±9.58e−07)(−)	**0.00e−00 (±1.78e−10)(1.08e−08)**	**0.00e+00 (±0.00e+00)(1.08e−08)**
f_7	**0.00e+00 (±0.00e+00)**(−)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_8	1.04e−08 (±7.86e−10)(−)	**0.00e+00 (±0.00e+00)(2.19e−14)**	**0.00e+00 (±0.00e+00)(2.19e−14)**
f_9	**1.42e+01 (±1.63e−01)**(−)	3.05e+01 (±1.88e−01)(6.99e−15)	2.97e+01 (±2.56e−01)(6.65e−11)
f_{10}	**2.85e−01 (±3.09e−03)**(−)	1.11e+01 (±4.74e−02)(5.14e−12)	9.99e+00 (±5.01e−03)(5.78e−06)
f_{11}	**2.68e+03 (±2.62e+01)**(−)	4.25e+03 (±1.83e+01)(2.19e−14)	4.97e+03 (±1.19e+01)(0.00e+00)
f_{12}	1.60e−01 (±1.13e−03)(−)	**5.16e−02 (±2.13e−03)(0.00e+00)**	6.77e−02 (±1.42e−03)(2.46e−11)
f_{13}	**1.17e−01 (±1.17e−03)**(−)	1.23e−01 (±1.15e−03)(4.03e−03)	1.21e−01 (±1.76e−03)(7.85e−03)
f_{14}	**2.59e−01 (±4.76e−04)**(−)	2.99e−01 (±1.52e−03)(3.02e−10)	2.96e−01 (±1.47e−03)(2.53e−07)
f_{15}	**4.69e+00 (±2.45e−02)**(−)	4.85e+00 (±1.74e−02)(1.48e−02)	4.84e+00 (±2.76e−02)(1.95e−03)
f_{16}	**1.57e+01 (±2.44e−02)**(−)	1.77e+01 (±3.65e−02)(1.11e−15)	1.80e+01 (±3.46e−02)(0.00e+00)
f_{17}	**3.48e+02 (±8.77e+00)**(−)	5.94e+02 (±1.37e+01)(5.14e−12)	4.75e+02 (±1.25e+01)(4.62e−05)
f_{18}	**9.00e+01 (±4.33e−01)**(−)	9.47e+01 (±6.23e−01)(8.81e−04)	1.03e+02 (±7.96e−01)(6.23e−13)
f_{19}	8.57e+00 (±9.36e−02)(−)	6.50e+00 (±2.43e−01)(2.41e−05)	**6.29e+00 (±1.62e−01)(6.23e−13)**
f_{20}	**8.06e+00 (±1.18e−01)**(−)	9.43e+00 (±1.16e−01)(2.06e−04)	1.08e+01 (±2.01e−01)(1.74e−12)
f_{21}	**3.57e+02 (±4.74e+00)**(−)	3.72e+02 (±4.47e+00)(1.20e−02)	3.88e+02 (±5.43e+00)(8.83e−05)
f_{22}	**1.36e+02 (±1.84e+00)**(−)	1.78e+02 (±1.23e+00)(2.03e−08)	1.83e+02 (±2.48e+00)(2.03e−08)
f_{23}	**1.24e+02 (±0.00e+00)**(−)	**1.24e+02 (±0.00e+00)(1.00e+00)**	**1.24e+02 (±0.00e+00)(1.00e+00)**
f_{24}	**2.70e+02 (±8.38e−02)**(−)	2.71e+02 (±1.74e−01)(4.03e−03)	2.71e+02 (±1.56e−01)(4.03e−03)
f_{25}	2.00e+02 (±2.19e−04)(−)	**2.00e+02 (±0.00e+00)(1.09e−02)**	2.00e+02 (±1.98e−04)(3.69e−02)
f_{26}	1.00e+02 (±6.96e−04)(−)	**1.00e+02 (±7.55e−04)(4.00e−04)**	1.00e+02 (±1.48e−03)(6.50e−01)
f_{27}	4.00e+02 (±3.21e−05)(−)	**4.00e+02 (±4.97e−05)(1.00e−04)**	4.00e+02 (±5.88e−05)(1.00e−04)
f_{28}	5.73e+02 (±4.29e−01)(−)	**5.71e+02 (±9.31e−01)(8.81e−04)**	**5.71e+02 (±9.19e−01)(3.73e−04)**
f_{29}	**7.48e+02 (±3.58e+00)**(−)	7.69e+02 (±2.19e+00)(4.03e−03)	7.90e+02 (±3.11e+00)(2.03e−08)
f_{30}	**6.23e+02 (±1.44e+00)**(−)	6.69e+02 (±2.83e+00)(4.05e−11)	6.38e+02 (±1.44e+00)(2.50e−03)

adopted to speed up convergence; and (3) a premature convergence detection method and a stagnation detection method are proposed to alleviate premature convergence and stagnation problems in the evolution by enhancing the diversity of the population. The experimental results showed that ADECSA has the best overall performance compared with other clonal algorithms and DE-based algorithms.

Table 15
Average error on D = 100. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error and the *p*-value, respectively.

Fun.	DDMS	FIX	PROB
f_1	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_2	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_3	**0.00e+00 (±0.00e+00)**(–)	**0.00e+00 (±0.00e+00)(1.00e+00)**	**0.00e+00 (±0.00e+00)(1.00e+00)**
f_4	**9.34e+01 (±2.50e+00)**(–)	**8.70e+01 (±1.38e+00)(5.00e−06)**	**8.68e+01 (±4.57e+00)(5.45e−05)**
f_5	2.04e+01 (±1.33e−02)(–)	**2.00e+01 (±1.69e−04)(2.46e−13)**	**2.00e+01 (±3.46e−04)(6.98e−06)**
f_6	**7.01e+00 (±6.11e−02)**(–)	4.59e+01 (±2.91e−01)(0.00e+00)	4.58e+01 (±6.20e−01)(0.00e+00)
f_7	**0.00e+00 (±0.00e+00)**(–)	2.53e−03 (±2.73e−04)(5.00e−06)	5.39e−03 (±6.62e−04)(0.00e+00)
f_8	2.86e−01 (±1.33e−02)(–)	**0.00e+00 (±0.00e+00)(0.00e+00)**	4.52e−01 (±1.17e−01)(1.88e−01)
f_9	**4.08e+01 (±4.02e−01)**(–)	1.05e+02 (±8.85e−01)(0.00e+00)	1.09e+02 (±1.70e+00)(0.00e+00)
f_{10}	5.52e+01 (±2.94e−01)(–)	**1.23e−02 (±9.50e−04)(0.00e+00)**	8.47e−02 (±1.08e−02)(1.39e−14)
f_{11}	9.62e+03 (±2.30e+01)(–)	**8.57e+03 (±2.63e+01)(5.14e−11)**	9.46e+03 (±7.27e+01)(1.23e−01)
f_{12}	3.33e−01 (±2.06e−03)(–)	**1.04e−01 (±1.47e−03)(1.39e−14)**	1.41e−01 (±2.35e−03)(1.29e−04)
f_{13}	**2.11e−01 (±5.07e−04)**(–)	3.42e−01 (±2.86e−03)(0.00e+00)	3.19e−01 (±3.15e−03)(7.33e−05)
f_{14}	**1.05e−01 (±6.07e−04)**(–)	1.14e−01 (±6.00e−04)(1.29e−04)	1.28e−01 (±6.27e−04)(0.00e+00)
f_{15}	**1.23e+01 (±1.61e−01)**(–)	2.03e+01 (±2.99e−01)(4.05e−08)	2.18e+01 (±4.41e−01)(1.66e−09)
f_{16}	3.76e+01 (±4.11e−02)(–)	**3.59e+01 (±5.49e−02)(4.27e−13)**	3.65e+01 (±1.24e−01)(1.29e−04)
f_{17}	**2.23e+03 (±2.01e+01)**(–)	3.78e+03 (±2.94e+01)(1.41e−10)	3.95e+03 (±7.92e+01)(6.66e−16)
f_{18}	**2.15e+02 (±6.50e−01)**(–)	2.35e+02 (±1.91e+00)(1.66e−09)	2.32e+02 (±1.47e+00)(1.46e−07)
f_{19}	1.04e+02 (±2.61e−01)(–)	9.06e+01 (±1.31e+00)(1.33e−15)	1.05e+02 (±1.22e+00)(4.82e−01)
f_{20}	**3.42e+01 (±2.54e−01)**(–)	1.54e+02 (±2.19e+00)(0.00e+00)	1.38e+02 (±2.50e+00)(1.62e−06)
f_{21}	**6.60e+02 (±9.26e+00)**(–)	1.81e+03 (±2.83e+01)(3.67e−12)	1.77e+03 (±2.58e+01)(3.11e−11)
f_{22}	8.88e+02 (±1.22e+01)(–)	**8.49e+02 (±8.26e+00)(8.75e−04)**	1.10e+03 (±2.27e+01)(1.29e−04)
f_{23}	**1.23e+02 (±0.00e+00)**(–)	**1.23e+02 (±0.00e+00)(1.00e+00)**	1.40e+02 (±3.44e+00)(1.30e−12)
f_{24}	**3.71e+02 (±1.02e−01)**(–)	3.76e+02 (±3.50e−01)(3.96e−05)	3.79e+02 (±3.97e−01)(1.41e−13)
f_{25}	**2.01e+02 (±8.84e−04)**(–)	2.01e+02 (±1.19e−03)(5.09e−04)	2.01e+02 (±1.27e−03)(1.67e−12)
f_{26}	**1.00e+02 (±1.32e−03)**(–)	1.00e+02 (±3.62e−03)(0.00e+00)	1.00e+02 (±3.47e−03)(2.16e−12)
f_{27}	**4.00e+02 (±3.75e−05)**(–)	4.00e+02 (±2.17e−04)(0.00e+00)	4.00e+02 (±2.56e−04)(0.00e+00)
f_{28}	**8.31e+02 (±1.85e+00)**(–)	1.03e+03 (±4.92e+00)(0.00e+00)	1.04e+03 (±9.38e+00)(0.00e+00)
f_{29}	**7.13e+02 (±1.36e+00)**(–)	8.19e+02 (±7.51e+00)(7.33e−05)	9.83e+02 (±2.41e+01)(0.00e+00)
f_{30}	**2.15e+03 (±1.26e+01)**(–)	3.12e+03 (±1.96e+01)(4.55e−15)	3.31e+03 (±3.45e+01)(0.00e+00)

Table 16
Average error on D = 50: DDMS *vs.* some recent sequential EAs. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error.

Fun.	DDMS	L-SHADE EpSin [47]	FLDE [48]	MSHCSA [49]	ADECSA [50]
f_1	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	1.05e+03 (±1.87e+03)
f_2	**0.00e+00 (±0.00e+00)**	1.57e+00 (±1.93e+00)	**0.00e+00 (±0.00e+00)**	2.51e−07 (±1.77e−07)	3.31e−08 (±9.65e−08)
f_3	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**
f_4	**1.36e+01 (±1.10e+00)**	5.14e+01 (±4.43e+01)	5.77e+01 (±5.05e+01)	2.72e+01 (±8.12e−01)	5.30e+01 (±4.83e+01)
f_5	2.02e+01 (±3.09e−03)	2.52e+01 (±6.44e+00)	**1.14e+01 (±2.61e+00)**	2.00e+01 (±2.44e−04)	2.02e+01 (±1.35e−01)
f_6	1.27e−05 (±9.58e−07)	9.16e−07 (±1.07e−06)	**0.00e+00 (±0.00e+00)**	5.09e+01 (±1.79e+00)	3.46e−02 (±1.29e−01)
f_7	**0.00e+00 (±0.00e+00)**	7.66e+01 (±6.06e+00)	6.32e+01 (±1.71e+00)	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**
f_8	1.04e−08 (±7.86e−10)	2.63e+01 (±6.59e+00)	1.13e+01 (±2.14e+00)	3.26e+01 (±3.80e+00)	**0.00e+00 (±0.00e+00)**
f_9	1.42e+01 (±1.63e−01)	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	7.68e+01 (±8.82e+00)	2.28e+01 (±9.51e+00)
f_{10}	2.85e−01 (±3.09e−03)	3.20e+03 (±3.40e+02)	3.04e+03 (±3.37e+02)	1.51e+03 (±2.24e+02)	**4.77e−02 (±1.65e−01)**
f_{11}	2.68e+03 (±2.62e+01)	**2.14e+01 (±2.09e+00)**	4.92e+01 (±1.07e+01)	5.79e+03 (±4.10e+02)	3.64e+03 (±5.63e+02)
f_{12}	**1.60e−01 (±1.13e−03)**	1.48e+03 (±3.65e+02)	2.33E+ 03 (±5.97E+ 02)	6.82e−01 (±1.02e−01)	1.84e−01 (±7.37e−02)
f_{13}	**1.17e−01 (±1.17e−03)**	6.94e+01 (±3.45e+01)	6.86e+01 (±3.24e+01)	2.84e−01 (±2.99e−02)	1.73e−01 (±2.49e−02)
f_{14}	2.59e−01 (±4.76e−04)	2.65e+01 (±2.49e+00)	3.02E+ 01 (±3.47E+ 00)	2.22e−01 (±4.02e−02)	**1.81e−01 (±2.54e−02)**
f_{15}	**4.69e+00 (±2.45e−02)**	2.56e+01 (±4.06e+00)	4.07e+01 (±1.01e+01)	9.19e+00 (±1.24e+00)	5.16e+00 (±5.11e−01)
f_{16}	**1.57e+01 (±2.44e−02)**	2.75e+02 (±9.97e+01)	3.92e+02 (±1.20e+02)	2.03e+01 (±3.24e−01)	1.66e+01 (±1.01e+00)
f_{17}	3.48e+02 (±8.77e+00)	2.77e+02 (±7.31e+01)	**2.49e+02 (±7.94e+01)**	2.64e+03 (±2.99e+02)	4.70e+02 (±1.87e+02)
f_{18}	9.00e+01 (±4.33e−01)	2.43e+01 (±2.12e+00)	4.52e+01 (±1.46e+01)	7.78e+01 (±1.02e+01)	**2.17e+01 (±6.91e+00)**
f_{19}	8.57e+00 (±9.36e−02)	1.74e+01 (±2.47e+00)	2.70e+01 (±9.17e+00)	1.96e+01 (±1.56e+00)	**8.45e+00 (±1.40e+00)**
f_{20}	**8.06e+00 (±1.18e−01)**	1.14e+02 (±3.55e+01)	1.63e+02 (±6.98e+01)	4.89e+01 (±5.76e+00)	8.63e+00 (±2.46e+00)
f_{21}	3.57e+02 (±4.74e+00)	2.27e+02 (±7.06e+00)	**2.14e+02 (±2.29e+00)**	1.50e+03 (±2.48e+02)	4.06e+02 (±1.49e+02)
f_{22}	1.36e+02 (±1.84e+00)	1.59e+03 (±1.67e+03)	1.13e+03 (±1.65e+03)	4.63e+02 (±1.32e+02)	**1.09e+02 (±8.99e+01)**
f_{23}	**1.24e+02 (±0.00e+00)**	4.39e+02 (±6.90e+00)	4.31e+02 (±3.74e+00)	3.37e+02 (±2.85e−13)	2.00e+02 (±0.00e+00)
f_{24}	2.70e+02 (±8.38e−02)	5.13e+02 (±5.59e+00)	5.09e+02 (±2.60e+00)	2.64e+02 (±3.30e−01)	**2.00e+02 (±2.45e−13)**
f_{25}	**2.00e+02 (±2.19e−04)**	4.80e+02 (±1.08e+00)	4.83e+02 (±5.90e+00)	**2.00e+02 (±1.59e−02)**	**2.00e+02 (±0.00e+00)**
f_{26}	**1.00e+02 (±6.96e−04)**	1.20e+03 (±1.19e+02)	1.18e+03 (±4.45e+01)	**1.00e+02 (±2.83e−02)**	**1.00e+02 (±2.45e−02)**
f_{27}	4.00e+02 (±3.21e−05)	5.25e+02 (±9.21e+00)	5.24e+02 (±8.85e+00)	1.73e+03 (±3.70e+01)	**2.00e+02 (±8.34e−10)**
f_{28}	5.73e+02 (±4.29e−01)	4.59e+02 (±1.19e+01)	4.59e+02 (±3.81e−02)	3.57e+02 (±4.41e−01)	**2.00e+02 (±0.00e+00)**
f_{29}	7.48e+02 (±3.58e+00)	3.53e+02 (±9.78e+00)	3.48e+02 (±9.74e+00)	2.29e+02 (±8.98e−01)	**2.00e+02 (±0.00e+00)**
f_{30}	6.23e+02 (±1.44e+00)	6.58e+05 (±7.24e+04)	6.14e+05 (±4.38e+04)	9.58e+02 (±1.77e+02)	**2.00e+02 (±0.00e+00)**

Table 17
Average error on D = 100: DDMS *vs.* some recent sequential EAs. Numbers in bold highlight represent the strategies with the lowest average error. Numbers in parentheses represent the standard error.

Fun.	DDMS	L-SHADE EpSin [47]	FLDE [48]	MSHCSA [49]	ADECSA [50]
f_1	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	9.95e+04 (±4.26e+04)	1.40e+05 (±4.94e+04)
f_2	**0.00e+00 (±0.00e+00)**	3.57e+11 (±6.18e+11)	2.35e+11 (±5.10e+11)	2.46e+00 (±1.21e+00)	4.60e−03 (±4.99e−03)
f_3	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	6.59e−07 (±3.64e−07)	**0.00e+00 (±0.00e+00)**
f_4	9.34e+01 (±2.50e+00)	1.99e+02 (±8.30e+00)	2.01e+02 (±7.59e+00)	**8.44e+01 (±6.18e−01)**	1.82e+02 (±2.97e+01)
f_5	2.04e+01 (±1.33e−02)	5.59e+01 (±9.91e+00)	3.55e+01 (±5.58e+00)	**2.00e+01 (±1.87e−03)**	2.05e+01 (±1.74e−01)
f_6	7.01e+00 (±6.11e−02)	6.02e−05 (±2.18e−05)	**0.00e+00 (±0.00e+00)**	1.29e+02 (±2.61e+00)	5.07e−01 (±6.34e−01)
f_7	**0.00e+00 (±0.00e+00)**	1.62e+02 (±7.91e+00)	1.41e+02 (±5.48e+00)	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**
f_8	2.86e−01 (±1.33e−02)	5.35e+01 (±5.39e+00)	3.55e+01 (±3.64e+00)	2.36e+02 (±1.64e+01)	**8.78e−06 (±1.78e−05)**
f_9	4.08e+01 (±4.02e−01)	**0.00e+00 (±0.00e+00)**	**0.00e+00 (±0.00e+00)**	3.17e+02 (±1.82e+01)	4.78e+01 (±1.66e+01)
f_{10}	5.52e+01 (±2.94e−01)	1.03e+04 (±5.21e+02)	1.07e+04 (±4.74e+02)	1.11e+04 (±5.54e+02)	**1.13e+00 (±5.21e−01)**
f_{11}	9.62e+03 (±2.30e+01)	**4.92e+01 (±3.02e+02)**	4.34e+02 (±1.31e+02)	1.69e+04 (±5.54e+02)	1.01e+04 (±1.10e+03)
f_{12}	3.33e−01 (±2.06e−03)	4.62e+03 (±6.48e+02)	2.34e+04 (±8.91e+03)	1.33e+00 (±1.34e−01)	**3.31e−01 (±4.74e−02)**
f_{13}	**2.11e−01 (±5.07e−04)**	1.25e+02 (±3.65e+01)	5.61e+02 (±2.73e+02)	3.67e−01 (±3.12e−02)	3.05e−01 (±2.58e−02)
f_{14}	**1.05e−01 (±6.07e−04)**	4.97e+01 (±8.17e+00)	2.45e+02 (±3.34e+01)	3.12e−01 (±9.89e−02)	2.19e−01 (±1.63e−02)
f_{15}	**1.23e+01 (±1.61e−01)**	8.99e+01 (±2.83e+01)	2.37e+02 (±6.18e+01)	3.00e+01 (±1.84e+00)	1.54e+01 (±9.47e−01)
f_{16}	**3.76e+01 (±4.11e−02)**	1.82e+03 (±2.36e+02)	1.53e+03 (±2.50e+02)	4.35e+01 (±4.63e−01)	3.84e+01 (±6.95e−01)
f_{17}	2.23e+03 (±2.01e+01)	1.32e+03 (±1.74e+02)	**1.24e+03 (±1.26e+02)**	5.87e+03 (±6.98e+02)	2.99e+03 (±5.52e+02)
f_{18}	2.15e+02 (±6.50e−01)	**7.79e+01 (±1.99e+01)**	1.97e+02 (±3.95e+01)	2.82e+02 (±9.44e+01)	1.46e+02 (±2.39e+01)
f_{19}	1.04e+02 (±2.61e−01)	9.55e+01 (±6.05e+00)	1.68e+02 (±1.52e+01)	**4.31e+01 (±3.35e−01)**	8.89e+01 (±1.33e+00)
f_{20}	3.42e+01 (±2.54e−01)	1.08e+03 (±2.16e+02)	1.59e+03 (±1.70e+02)	2.18e+02 (±1.49e+01)	**3.16e+01 (±5.79e+00)**
f_{21}	6.60e+02 (±9.26e+00)	2.77e+02 (±6.94e+00)	**2.60e+02 (±4.13e+00)**	4.76e+03 (±3.41e+02)	6.30e+02 (±2.55e+02)
f_{22}	**8.88e+02 (±1.22e+01)**	1.04e+04 (±5.30e+02)	1.14e+04 (±6.65e+02)	1.53e+03 (±3.35e+02)	9.52e+02 (±2.50e+02)
f_{23}	**1.23e+02 (±0.00e+00)**	5.98e+02 (±7.69e+00)	5.68e+02 (±7.79e+00)	3.45e+02 (±1.05e−12)	2.00e+02 (±0.00e+00)
f_{24}	3.71e+02 (±1.02e−01)	9.17e+02 (±1.34e+01)	9.14e+02 (±8.57e+00)	3.84e+02 (±2.91e+00)	**2.00e+02 (±2.45e−13)**
f_{25}	2.01e+02 (±8.84e−04)	6.84e+02 (±4.34e+01)	7.34e+02 (±4.18e+01)	2.01e+02 (±1.70e−01)	**2.00e+02 (±0.00e+00)**
f_{26}	**1.00e+02 (±1.32e−03)**	3.11e+03 (±1.22e+02)	3.34e+03 (±1.07e+02)	**1.00e+02 (±2.89e−02)**	**1.00e+02 (±2.45e−02)**
f_{27}	4.00e+02 (±3.75e−05)	5.89e+02 (±1.31e+01)	6.40e+02 (±2.17e+01)	3.71e+03 (±5.61e+01)	**2.00e+02 (±7.83e−10)**
f_{28}	8.31e+02 (±1.85e+00)	5.25e+02 (±2.20e+01)	5.24e+02 (±1.36e+01)	4.04e+02 (±5.30e+00)	**2.00e+02 (±0.00e+00)**
f_{29}	7.13e+02 (±1.36e+00)	1.12e+03 (±1.49e+02)	1.32e+03 (±1.95e+02)	2.43e+02 (±4.22e+00)	**2.00e+02 (±0.00e+00)**
f_{30}	2.15e+03 (±1.26e+01)	2.46e+03 (±1.44e+02)	2.41e+03 (±1.78e+02)	2.85e+03 (±2.73e+02)	**2.00e+02 (±0.00e+00)**

Considering the better performance of the DDMS in scenarios with a greater number of variables, the error values ($f(\mathbf{x}) - f(\mathbf{x}^*)$) are given in Tables 16 and 17 for 50D and 100D, respectively. The numbers in parentheses represent the standard error of each algorithm. The results of the LSHADE EpSin, FLDE, MSHCSA, and ADECSA are extracted from [48,50], respectively. We added a bold highlight to the strategies that obtained the lowest average error for each function.

From results shown in Table 16, we can find that DDMS wins on 18 functions when compared with DE-based algorithms (L-SHADE EpSin and FLDE). Only on 9 occasions DDMS is overcome by L-SHADE EpSin or FLDE. Table 17 shows the corresponding results on 100D. For the average performance, it wins on 19 benchmarks over all other DE-based algorithms. These results demonstrate that DDMS has the potential to improve L-SHADE and make it outperform recent DE-based sequential algorithms.

When compared with recent clonal selection algorithms (MSHCSA and ADECSA), DDMS wins on 13 and 11 functions considering 50 and 100 dimensions, respectively. Regarding the recent ADECSA, which demonstrated excellent performance in [50] when compared with DE-based and clonal selection algorithms, DDMS loses on 12 and 16 functions and wins in 13 and 11 functions, considering 50D and 100D, respectively. Again, these results show the potential of DDMS to improve the performance of L-SHADE.

The results demonstrate that the DDMS can promote better performance of evolutionary algorithms within a distributed model. In future work, we want to test it using more recent optimizers, such as FLDE and ADECSA, in order to validate this hypothesis.

References

[1] Michel Gendreau, Jean-Yves Potvin, et al., Handbook of Metaheuristics, Vol. 2, Springer, 2010.

[2] Yue-Jiao Gong, Wei-Neng Chen, Zhi-Hui Zhan, Jun Zhang, Yun Li, Qingfu Zhang, Jing-Jing Li, Distributed evolutionary algorithms and their models: A survey of the state-of-the-art, Appl. Soft Comput. 34 (2015) 286–300.

[3] Takashi Ishimizu, Kiyoharu Tagawa, A structured differential evolution for various network topologies, Int. J. Comput. Commun. 4 (1) (2010) 2–8.

[4] Yann Lorion, Tjorben Bogon, Ingo J. Timm, Oswald Drobnik, An agent based parallel particle swarm optimization-APPSO, in: 2009 IEEE Swarm Intelligence Symposium, IEEE, 2009, pp. 52–59.

[5] Chunmei Zhang, Jie Chen, Bin Xin, Distributed memetic differential evolution with the synergy of Lamarckian and Baldwinian learning, Appl. Soft Comput. 13 (5) (2013) 2947–2959.

[6] Javier Apolloni, José García-Nieto, Enrique Alba, Guillermo Leguizamón, Empirical evaluation of distributed differential evolution on standard benchmarks, Appl. Math. Comput. 236 (2014) 351–366.

[7] Plamen Angelov, Outside the box: an alternative data analytics framework, J. Autom. Mobile Robot. Intell. Syst. 8 (2) (2014) 29–35.

[8] A.E. Eiben, James E. Smith, Introduction To Evolutionary Computing, second ed., Springer Publishing Company, Incorporated, 2015.

[9] Alain Pétrowski, Sana Ben-Hamida, Evolutionary Algorithms, John Wiley & Sons, 2017.

[10] Matej Črepinšek, Shih-Hsi Liu, Marjan Mernik, Exploration and exploitation in evolutionary algorithms: A survey, ACM Comput. Surv. 45 (3) (2013) 35.

[11] Giorgos Karafotias, Mark Hoogendoorn, Ágoston E. Eiben, Parameter control in evolutionary algorithms: Trends and challenges, IEEE Trans. Evol. Comput. 19 (2) (2014) 167–187.

[12] Nandar Lynn, Ponnuthurai Nagaratnam Suganthan, Heterogeneous comprehensive learning particle swarm optimization with enhanced exploration and exploitation, Swarm Evol. Comput. 24 (2015) 11–24.

[13] Mostafa Z Ali, Noor H Awad, Ponnuthurai Nagaratnam Suganthan, Robert G Reynolds, An adaptive multipopulation differential evolution with dynamic population reduction, IEEE Trans. Cybern. 47 (9) (2016) 2768–2779.

[14] Laizhong Cui, Genghui Li, Qiuzhen Lin, Jianyong Chen, Nan Lu, Adaptive differential evolution algorithm with novel mutation strategies in multiple sub-populations, Comput. Oper. Res. 67 (2016) 155–173.

[15] Dirk Sudholt, The benefits of population diversity in evolutionary algorithms: a survey of rigorous runtime analyses, in: Theory of Evolutionary Computation, Springer, 2020, pp. 359–404.

[16] Dongyang Li, Weian Guo, Alexander Lerch, Yongmei Li, Lei Wang, Qidi Wu, An adaptive particle swarm optimizer with decoupled exploration and exploitation for large scale optimization, Swarm Evol. Comput. 60 (2021) 100789.

[17] Ming Yang, Changhe Li, Zhihua Cai, Jing Guan, Differential evolution with auto-enhanced population diversity, IEEE Trans. Cybern. 45 (2) (2014) 302–315.

[18] Brian McGinley, John Maher, Colm O'Riordan, Fearghal Morgan, Maintaining healthy population diversity using adaptive crossover, mutation, and selection, IEEE Trans. Evol. Comput. 15 (5) (2011) 692–714.

[19] Giovanni Squillero, Alberto Tonda, Divergence of character and premature convergence: A survey of methodologies for promoting diversity in evolutionary optimization, Inform. Sci. 329 (2016) 782–799.
[20] Rodolfo A. Lopes, Alan R.R. de Freitas, Island-cellular model differential evolution for large-scale global optimization, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion, 2017, pp. 1841–1848.
[21] Ting-Chen Wang, Chih-Yu Lin, Rung-Tzuo Liaw, Chuan-Kang Ting, Empirical analysis of island model on large scale global optimization, in: 2019 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2019, pp. 342–349.
[22] Matthieu Weber, Ferrante Neri, Ville Tirronen, Distributed differential evolution with explorative–exploitative population families, Genet. Program. Evol. Mach. 10 (4) (2009) 343.
[23] Lourdes Araujo, Juan Julián Merelo, Diversity through multiculturality: Assessing migrant choice policies in an island model, IEEE Trans. Evol. Comput. 15 (4) (2010) 456–469.
[24] Qinxue Meng, Jia Wu, John Ellis, Paul J. Kennedy, Dynamic island model based on spectral clustering in genetic algorithm, in: 2017 International Joint Conference on Neural Networks (IJCNN), IEEE, 2017, pp. 1724–1731.
[25] Grasiele Regina Duarte, Afonso Celso de Castro Lemonge, Leonardo Goliatt da Fonseca, Beatriz Souza Leite Pires de Lima, An Island Model based on Stigmergy to solve optimization problems, Nat. Comput. 20 (3) (2021) 413–441.
[26] Jiawei Li, Tad Gonsalves, Parallel hybrid Island metaheuristic algorithm, IEEE Access 10 (2022) 42268–42286.
[27] Alfian Akbar Gozali, Shigeru Fujimura, DM-LIMGA: dual migration localized island model genetic algorithm—a better diversity preserver island model, Evol. Intell. 12 (4) (2019) 527–539.
[28] Adam P. Piotrowski, Jaroslaw J. Napiorkowski, Adam Kiczko, Differential evolution algorithm with separated groups for multi-dimensional optimization problems, European J. Oper. Res. 216 (1) (2012) 33–46.
[29] Yong-Feng Ge, Wei-Jie Yu, Ying Lin, Yue-Jiao Gong, Zhi-Hui Zhan, Wei-Neng Chen, Jun Zhang, Distributed differential evolution based on adaptive mergence and split for large-scale optimization, IEEE Trans. Cybern. 48 (7) (2017) 2166–2180.
[30] Ivanoe De Falco, Umberto Scafuri, Ernesto Tarantino, Antonio Della Cioppa, A distributed differential evolution approach for mapping in a grid environment, in: 15th EUROMICRO International Conference on Parallel, Distributed and Network-Based Processing (PDP'07), IEEE, 2007, pp. 442–449.
[31] Mostafa Z. Ali, Noor H. Awad, Ponnuthurai N. Suganthan, Multi-population differential evolution with balanced ensemble of mutation strategies for large-scale global optimization, Appl. Soft Comput. 33 (2015) 304–327.
[32] Adam P. Piotrowski, Jaroslaw J. Napiorkowski, Step-by-step improvement of JADE and SHADE-based algorithms: Success or failure? Swarm Evol. Comput. 43 (2018) 88–108.
[33] Jarosław Arabas, Karol Opara, Population diversity of nonelitist evolutionary algorithms in the exploration phase, IEEE Trans. Evol. Comput. 24 (6) (2019) 1050–1062.
[34] Sedigheh Mahdavi, Mohammad Ebrahim Shiri, Shahryar Rahnamayan, Metaheuristics in large-scale global continues optimization: A survey, Inform. Sci. 295 (2015) 407–428.
[35] Antonio LaTorre, Santiago Muelas, José-María Peña, A comprehensive comparison of large scale global optimizers, Inform. Sci. 316 (2015) 517–549.
[36] Clauber Gomes Bezerra, Bruno Sielly Jales Costa, Luiz Affonso Guedes, Plamen Parvanov Angelov, A new evolving clustering algorithm for online data streams, in: 2016 IEEE Conference on Evolving and Adaptive Intelligent Systems (EAIS), IEEE, 2016, pp. 162–168.
[37] José Maia, Carlos Alberto Severiano Junior, Frederico Gadelha Guimarães, Cristiano Leite de Castro, André Paim Lemos, Juan Camilo Fonseca Galindo, Miri Weiss Cohen, Evolving clustering algorithm based on mixture of typicalities for stream data mining, Future Gener. Comput. Syst. 106 (2020) 672–684.
[38] John G. Saw, Mark C.K. Yang, Tse Chin Mo, Chebyshev inequality with estimated mean and variance, Amer. Statist. 38 (2) (1984) 130–132.
[39] Ryoji Tanabe, Alex S. Fukunaga, Improving the search performance of SHADE using linear population size reduction, in: 2014 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2014, pp. 1658–1665.
[40] Jing J. Liang, Bo Y. Qu, Ponnuthurai N. Suganthan, Problem definitions and evaluation criteria for the CEC 2014 special session and competition on single objective real-parameter numerical optimization, in: Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou China and Technical Report, Vol. 635, Nanyang Technological University, Singapore, 2013.
[41] Ryoji Tanabe, Alex Fukunaga, Success-history based parameter adaptation for differential evolution, in: 2013 IEEE Congress on Evolutionary Computation, IEEE, 2013, pp. 71–78.
[42] Daniel Molina, Antonio LaTorre, Francisco Herrera, SHADE with iterative local search for large-scale global optimization, in: 2018 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2018, pp. 1–8.
[43] Jingqiao Zhang, Arthur C. Sanderson, JADE: adaptive differential evolution with optional external archive, IEEE Trans. Evol. Comput. 13 (5) (2009) 945–958.
[44] Juan Luís J Laredo, Carlos Fernandes, Juan Julián Merelo, Christian Gagné, Improving genetic algorithms performance via deterministic population shrinkage, in: Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation, 2009, pp. 819–826.
[45] Qiang Yang, Wei-Neng Chen, Jeremiah Da Deng, Yun Li, Tianlong Gu, Jun Zhang, A level-based learning swarm optimizer for large-scale optimization, IEEE Trans. Evol. Comput. 22 (4) (2017) 578–594.
[46] Ashwani Kumar Thukral, Renu Bhardwaj, Vinod Kumar, Anket Sharma, New indices regarding the dominance and diversity of communities, derived from sample variance and standard deviation, Heliyon 5 (10) (2019) e02606.
[47] Noor H Awad, Mostafa Z Ali, Ponnuthurai N Suganthan, Robert G Reynolds, An ensemble sinusoidal parameter adaptation incorporated with L-SHADE for solving CEC2014 benchmark problems, in: 2016 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2016, pp. 2958–2965.
[48] Zhiping Tan, Kangshun Li, Yi Wang, Differential evolution with adaptive mutation strategy based on fitness landscape analysis, Inform. Sci. 549 (2021) 142–163.
[49] Weiwei Zhang, Kui Gao, Weizheng Zhang, Xiao Wang, Qiuwen Zhang, Hua Wang, A hybrid clonal selection algorithm with modified combinatorial recombination and success-history based adaptive mutation for numerical optimization, Appl. Intell. 49 (2019) 819–836.
[50] Yi Wang, Tao Li, Xiaojie Liu, Jian Yao, An adaptive clonal selection algorithm with multiple differential evolution strategies, Inform. Sci. 604 (2022) 142–169.